

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 13.09.2026, 14:14:42
Файлов исходного кода: 87
Суммарный объём кода: 1.23 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts

25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts

27. src/data/tickets/ticket14_20.ts

28. src/data/tickets/ticket21_24.ts

29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx

31. src/components/BellmanFordViz.tsx

32. src/components/BfsViz.tsx

33. src/components/ChapterImage.tsx

34. src/components/ChapterNav.tsx

35. src/components/ChapterView.tsx

36. src/components/DfsBridgesSimulator.tsx

37. src/components/DijkstraViz.tsx

38. src/components/DPVisualizer.tsx

39. src/components/EulerSimulator.tsx

40. src/components/ExpressQuiz.tsx

41. src/components/FloydViz.tsx

42. src/components/GraphTraversalViz.tsx

43. src/components/HeapViz.tsx

44. src/components/hints/demoKit.tsx

45. src/components/hints/demosExtra.tsx

46. src/components/hints/demosGraph.tsx

47. src/components/hints/demosStruct.tsx

48. src/components/hints/MiniDemo.tsx

49. src/components/hints/SimulatorModal.tsx

50. src/components/hints/TermPopover.tsx

51. src/components/JohnsonViz.tsx

52. src/components/KosarajuViz.tsx

53. src/components/KruskalSimulator.tsx

54. src/components/MnemonicCards.tsx

55. src/components/MobileToc.tsx

56. src/components/Navbar.tsx

57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

```
 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```



```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full_code_all_pages
 echo "| \`public/export/full_code_all_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from
import { chapters } from "../data/content";
```



```

 }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
)}
 </Navbar>
 </div>
);

```

```

 </footer>
 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (
    <main>
      <ChapterNav />
      <ChapterView />
    </main>
  )
}

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-800" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-400 hover:text-slate-200 transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-400 hover:text-slate-200 transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

-----  
### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
    -none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

  <div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
      Ввод для input() • по одной строке на к
    </label>
    <textarea
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.tar
      spellCheck={false}
      rows={2}
      placeholder={'Например:\nАлиса'}
      className="block w-full resize-y bg-tra
    eholder:text-slate-700"
    />
  </div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
  x] leading-6 text-slate-300">
    {output}
  </pre>

```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {  
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             <label className="block px-4 pt-3 text-slate-600 font-medium">Ввод для input() • по одной строке на строку</label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-transparent border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             </div>
+         </section>
+
+         <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 20px; margin: 10px 0;}}>
+             <div className="flex items-center justify-between" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin: 10px 0;}}>
+                 <div className="flex items-center gap-2" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin: 10px 0;}}>
+                     <Terminal className="w-4 h-4 text-emerald-500 font-mono" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+                     </div>
+                     <span className={`inline-flex items-center gap-2`} style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin: 10px 0;}}>
+                         {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+                         {runtimeReady ? "готов" : "не загружен"}
+                     </span>
+                 </div>
+                 <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+                     {output}
+                 </pre>
+             </div>
+         </section>
+     </div>
+
+     <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin: 10px 0;}}>
+         <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>

```

```
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний прокси
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локально в /
+ // VITE_BASE можно переопределить, если репозиторий не на GitHub
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {

```

2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь структуру `{ "content": ... }` для вставки в `src/data/content.ts`.
3. **\*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, `bg-indigo-100`)
  - Определение/правило в центре (`border-blue-500` или `border-blue-500/2px`)
  - Обязательная сетка аналогий (2 колонки: неформальный и формальный язык)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы из `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/87: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТР0К: 14 | РАЗМЕР: 6

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, >
 <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
 <title> Дискретка для тупых – Интерактивный гид по
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen" >
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/87: metadata.ison

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 31

```
{
 "name": "Remix Remix: ExamPrep Reader",

```

```
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

---

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5174
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- \* **\*\*Содержание\*\*** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

## ## Структура

...

|                    |                                          |
|--------------------|------------------------------------------|
| src/               |                                          |
| App.tsx            | – оболочка, привязка глав к визуалу      |
| components/        | – интерактивные симуляторы (Дейкстра)    |
| data/              | – контент пособия (главы/билеты)         |
| scripts/export/    | – пайплайн код → txt → png → pdf         |
| public/export/     | – сгенерированные TXT и PDF (обновляемо) |
| .github/workflows/ | – автоматизация                          |
| ...                |                                          |

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```

> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет         | Тема                           | Статус                                    |
|---------------|--------------------------------|-------------------------------------------|
| ---           | ---                            | ---                                       |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую.            |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; есть блок «Аналогия». |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть блок «Аналогия». |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть блок «Аналогия». |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динамическое программирование\*\***

|    |                                               |   |   |
|----|-----------------------------------------------|---|---|
| 9  | 9. Графы. Топологическая сортировка           | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | 1 | - |
| 12 | 12. Графы. Точки сочленения                   | 1 | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 | - |
| 16 | 16. Графы. Кратчайший путь. Флойд             | - | - |
| 17 | 17. Графы. Алгоритм Джонсона                  | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 | - |
| 21 | 21. Префикс-функция (π), КМП                  | 4 | - |
| 22 | 22. Z-функция                                 | 4 | - |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 | - |
| 24 | 24. Классы сложности, сведение задач          | 4 | - |
| -  | Обзор: Кратчайшие пути и Графы                | - | - |
| -  | Алгоритмы на карте                            | - | - |
| -  | Эйлер: Путь ≠ Цикл                            | - | - |
| -  | Мосты в графах: код + визуализация            | 1 | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inline`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/87: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
```



```

import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

 if (compilerHeight !== null) window.localStorage.set
 }, [compilerHeight]);
 /** Содержание свернуто (по умолчанию сворачиваем сам
const [sidebarCollapsed, setSidebarCollapsed] = useSt
 const cached = typeof window !== "undefined" ? wind
 if (cached !== null) return cached === "1";
 return typeof window !== "undefined" && window.inne
});
const toggleSidebar = useCallback(() => {
 setSidebarCollapsed((c) => {
 window.localStorage.setItem("sidebarCollapsed", c
 return !c;
 });
}, []);

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.inn
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);

```

```

<div ref={contentRef} className={compilerOpen ? "
 {activeTab === "guide" ? (

```

```

 <>

```

```

 <MobileToc

```

```

 selectedChapterId={activeChapterId}

```

```

 setSelectedChapterId={goTo}

```

```

 currentTitle={activeChapter.title}

```

```

 index={index}

```

```

 total={chapters.length}

```

```

 />

```

```

 <div className="flex flex-col lg:flex-row m

```

```

 /* Сайдбар только на десктопе – на мобил

```

```

 <div

```

```

 className={`hidden lg:block shrink-0 bo
tion-all duration-200 ${sidebarCollapsed ?

```

```

 >

```

```

 {!sidebarCollapsed && <Sidebar selected

```

```

 </div>

```

```

 /* Вкладка-переключатель содержания у ле

```

```

 <button

```

```

 type="button"

```

```

 onClick={toggleSidebar}

```

```

 className="hidden lg:flex items-center
l-0 border-slate-700 bg-slate-900/90 px-1 p

```

```

 aria-label={sidebarCollapsed ? "Разверн

```

```

 title={sidebarCollapsed ? "Развернуть с

```

```

 >

```

```

 {sidebarCollapsed ? <PanelLeftOpen clas

```

```

 <span className="text-[10px] font-bold

```

```

 </button>

```

```

 <main id="main-content" className="flex-1

```

```

 /* Шапка главы с быстрыми переходами *

```

```

 <div className="flex items-center justifi

```

```

 <div className="min-w-0">

```

```

 <div className="text-[10px] upperca

```

```

 {activeChapter.category || "Универ

```

```
 </main>
 }}

 <footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивный
 </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 {compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
}}
</div>
);
}
```

---

ФАЙЛ 12/87: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
```

```

 return NodeFilter.FILTER_ACCEPT;
 },
 });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 }
}

```

```
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 К

---

```
@import "tailwindcss";
```

```
@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
 .no-scrollbar::-webkit-scrollbar {
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}
```

```
@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 }
}
```

```
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}
```

```
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```



```
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

## РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

---

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

---

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
 {
```

```
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
```

```
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C2][kx][ky]);
int minimum = min(ans1, ans2);</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "treap",
```

```
 title: "3. Декартово дерево (Treap)",
```

```
 type: "html",
```

```
 description: "Дерево поиска по ключу и Куча по приоритету",
```

```
 category: "Продвинутые структуры",
```

```
 content: `
```

```
<section id="treap" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
 Трешка
```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-white">3. Декартово дерево</h2>
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
```

```
 <h3 class="text-xl font-bold text-blue-400">Декартово дерево</h3>
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-blue-300 italic">Дерево поиска по ключу и Куча по приоритету</p>
```

```
 <p class="text-xl font-mono text-white">Аналогия: Бинарное дерево</p>
```

```
 метода: Split и Merge.</p>
```

```
 </div>
```

```
 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 right-4 bottom-4">
```

```
 <div class="border border-emerald-500 shadow-md">
```

```
 Аналогия 1: Удар катаной (Split по значению X)</div>
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Сортируем массив по значению X. Все, кто меньше X улетают в левое дерево, а кто больше X улетают в правое.</p>
```

```
 </div>
```

```
 </div>
```

```

 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <!-- 0. Определение -->
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-lg sm:text-xl font-mono">
 поиска без
 : серия поворотов, которая поднимает v в корень
 <p class="text-sm text-slate-400 text-cyan-300">
 т – поворот.</p>
 </div>

 <div class="grid grid-cols-1 lg:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt-2">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-emerald-300 font-bold">
 <p class="text-slate-400 text-xs mt-2">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">0(n

```

Аналогия 1: стопка бумаг на столе

```
</div>
<p class="text-slate-300 text-sm">У тебя
скиваешь его и, поработав, кладёшь
неделю самые нужные бумаги сами собой оказыва
ет ровно это: «взял → положил наверх».</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
```

Аналогия 2: тропинка через газон

```
</div>
<p class="text-slate-300 text-sm">Ты идёшь по дорожке, и дорожка сама укорачивалась вдвое: чем чаще ты её
оплачивает все следующие. Пойдёшь один раз
</div>
```

```
</div>
```

<!-- 3. Поворот -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border-rose-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-4">Поворот
 <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки и
 <pre class="bg-slate-950 p-4 rounded-lg overflow-hidden">
```

<pre>       /  \     20    C      /  \     A    B </pre>	<pre>       поворот 20     -----&gt;     &lt;-----       поворот 40 </pre>	<pre>       /  \     A    40      /  \     B    C </pre>
----------------------------------------------------------	----------------------------------------------------------------------------	----------------------------------------------------------

обход слева направо ДО: A 20 B 40 C

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же самый

```
<div class="grid grid-cols-1 md:grid-cols-2">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-emerald-400 font-bold">
 <p class="text-slate-300 text-xs">Поворот — это движение, три перевешенные ссылки
 сла поворотов — сломать его поворотами невозможно
 </div>
```

```

 </tr>
 <tr>
 <td class="py-3 pr-3 font-b<
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">р и г стал
 </tr>
 </tbody>
</table>
</div>

```

```

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

```

<!-- 5. Главная ловушка -->

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
 <h3 class="text-lg font-bold text-rose-300
 <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
 <div class="grid grid-cols-1 lg:grid-cols-2
 <div class="bg-slate-950 rounded-lg p-4
 <p class="text-rose-400 font-bold t
 <pre class="overflow-x-auto text-[1

```

```

 20
 /
 40
 /
20

 →

 20
 /
20
 \
 40

 →

 \
 60
 /
 40

```

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</pre>

```

 <p class="text-slate-400 text-xs mt
мортизации не получится.</p>
</div>

```

а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-1
 <p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
ах идея такая:</p>
 <div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">н
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">д
 </div>
 </div>
 <p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-1
 <p class="text-slate-300 text-sm mb-4">Крас
лается в корне.</p>
 <div class="space-y-2 text-sm">
 <div class="bg-slate-900 rounded-lg p-3
 — сныс
```

```

x.parent = g; // x занял его место
if (g) { if (g.left === p) g.left = x; else g.right =
}

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда нет
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
 }
 return x; // теперь x –
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
 let cur = root, last = null;
 while (cur) {
 last = cur;
 if (key === cur.key) break;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return last ? splay(last) : null; // splay делаем
}

```

`<p class="text-xs text-slate-400">Вся р`  
`ate(x);</span>` против `<span class="font-mon`  
`т свою оценку.</p>`

&lt;/details&gt;

```

 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Треар дер
 play не хранит вообще ничего и держит балан
 p>
 </div>
 </div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white"

```



```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #000; border-radius: 10px; margin: -3px 4px 4px 4px;">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-0" style="color: #a6c1ee; font-size: 0.8em; margin-bottom: 0;">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </p>
</div>
</div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 9. Графы. Топологическая сортировка
 <h2 class="text-2xl sm:text-3xl font-bold text-white">9. Графы. Топологическая сортировка</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Топологическая сортировка</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Топологическая сортировка — это алгоритм, который позволяет упорядочить вершины графа так, чтобы каждая вершина была размещена перед всеми вершинами, к которым она имеет рёбра.</p>
 <p class="text-xl font-mono text-white">Топологическая сортировка — это алгоритм, который позволяет упорядочить вершины графа так, чтобы каждая вершина была размещена перед всеми вершинами, к которым она имеет рёбра.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #000; border-radius: 10px; margin: -3px 4px 4px 4px;">

```

```

</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторон
 </div>
 <p class="text-slate-300 text-sm mb
БЫХ направлениях по односторонним улицам. К
у на карте.</p>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400 r
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic r
 <p class="text-xl font-mono text-white">
и $fup[v] > tin[u]$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg

```

```

 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb-4">
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]
```

```

 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-prima",

```

```

 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">

```

```

} </pre>
 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 щегося с i.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb">
 в тексте кричит: "Эй! Начиная с этой буквы
 т "окно" [L, R] самой дальней найденной коп
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче В, то В – это NP-Полная (сосредо
 </div>
</div>
</div>
</section>`,
 },
];

```

ФАЙЛ 18/87: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

```

export interface Chapter {
 id: string;
 title: string;

```

```

 финиш) .
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-900">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-slate-900 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная).
 Талантливый пришёл в бар (вторая нечётная).
 Домой вернуться не смог, потому что
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-950">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-slate-950 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.
 Гамильтон ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно раз.
 Главное – зайти и выйти, не заходя дважды.
 Никто не знает, как решать быстро.

 (Гамильтонов синдром – это болезнь, которая приводит к
 лноту).
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-950">
 <summary class="p-4 font-bold text-slate-400">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-4 text-slate-400">
 Душный режим: Формальное доказательство
 </div>
</details>

```



```
 <span class="text-rose-400 font-
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Если из сына и и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу
 Из пещеры и нет других выходов – то
 Если верёвка (ребро v-u) оборвётся -
 Если бы из пещеры и был чёрный ход
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
 der-rose-500 shadow-md">
 Аналогия: Кровеносный сосуд
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Граф – это кровеносная система. Реб
 терали) – это не мост, живём.
 Если же перерезать единственную арте
 Алгоритм DFS ищет такие «критически
 </p>
```

```

 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
}
}

```

```

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

```

</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">

```

КЗ,3: 3 дома и 3 колодца

</div>

<p class="text-slate-300 text-sm mb-4">

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него <code class="text-white"> $V=6$

<code class="text-white"> $9 \leq 12$ </code> – проходит, но он в

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: <code class="text-white">

<code class="text-white"> $9 \leq 8$ </code>

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-400">

order-slate-700/50">

Доказательство непланарности K5 (Скрыть)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-4">

<p class="text-blue-400">Доказательство

<p>

Пусть K5 планарен.  $V = 5$ ,  $E = 10$ .

<code> $F = E - V + 2 = 10 - 5 + 2 = 7$

<code>Каждая грань ограничена минимумом

<code>Сумма длин граней =  $2E = 20$ .

<code>Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  – <code>

<code>Значит, K5 не может быть планарным.

</p>

</div>

</details>

</div>

</section>`,

},

{

id: "graph-articulation",

title: "12. Графы. Точки сочленения",

type: "html",

её перекроют (удалят вершину) – районы буд  
>хрупкость</strong> системы.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-  
<div class="absolute -top-3 left-4 bg-em  
er border-emerald-500 shadow-md">

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной  
абелем (мостом). Сами свитчи – это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-  
<summary class="p-4 font-bold text-slate-400  
order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил  
то выше неё прыгать некуда). Поэтому для не  
у него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;
```

```
 for (int to : adj[v]) {
 if (to == p) continue;
```

---

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-bridges
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20)
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}
```

```

func: string;
/** Локальные переменные: имя → герг (усечённый), для
locals: Record<string, string>;
/**
 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
values?: Record<string, DebugValue>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Локалы в момент возврата из модуля – «результат»
finalLocals?: Record<string, string>;
 /** Финальный машинно-читаемый снимок для визуализации
finalValues?: Record<string, DebugValue>;
 /** Текст последней необработанной ошибки (или пустая
error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет user
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-с
 return `import sys as __sys, json as __json

__src = ${srcLiteral}
__steps = []
__final = {}
__final values = {}

```

```

 k: v for k, v in items
 if not k.startswith("__") and type(v).__name__
 }

def __capture(frame):
 local_values = __visible(frame.f_locals.items())
 # Внутри функции алгоритма важные структуры (memo,
 # глобальные. Локальные значения имеют приоритет на
 scope_values = __visible(frame.f_globals.items())
 scope_values.update(local_values)
 loc = {k: __safe(v) for k, v in local_values.items()}
 values = {k: __snapshot(v) for k, v in scope_values.items()}
 return loc, values

def __tracer(frame, event, arg):
 global __was_truncated
 if event == "line" and frame.f_code.co_filename == <
 and not (frame.f_code.co_name.startswith("<
 loc, values = __capture(frame)
 # PEP 709: с 3.12 компрехенشنы встроены во фреймворк
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __file__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

```

```
/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => na

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
}
```

---

ФАЙЛ 21/87: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 626 | РАЗМЕР: 1.5 КБ

---

```
import type { DemoKind } from "../components/hints/MiniHints";

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
 id: string;
 /** Варианты написания в тексте (регистр не важен). */
 labels: string[];
 explanation: string;
 visualization: string;
}
```



```

 },
 {
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого достаём элемент»",
 formal:
 "Полное бинарное дерево в массиве: дети узла i лежат в узлах $2i$ и $2i+1$.",
 complexity: "вставка/извлечение $O(\log n)$, «посмотреть элемент» $O(1)$ ",
 demo: "heap",
 simulator: "heap-beam-search",
 },
 {
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
 formal: "push / pop / top за $O(1)$. На стеке живёт рекурсия",
 demo: "stack",
 simulator: "stack-dfs",
 },
 {
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот первым и ест",
 formal: "push_back / pop_front за $O(1)$. Основа BFS и поиска в ширину",
 demo: "queue",
 simulator: "queue-bfs",
 },
 {
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся множеств"],
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего старосту",
 formal:
 "Система непересекающихся множеств (DSU) — это структура данных, которая позволяет эффективно управлять группами элементов, которые не пересекаются. Она используется для решения задач, связанных с объединением и поиском принадлежности элементов к той же группе."
 }
]
}

```

```

 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "то",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{

```

```

 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u, v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где low — минимальный индекс вершины в поддереве, достижимом из v по ребрам, не являющимся ребром в родителя — нет).",
 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точки"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения $\iff \exists$ ребро (u, v) такое, что $\text{low}[u] \geq \text{tin}[v]$ ",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная длина была минимальной",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно по возрастанию весов).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизированный",
],

```

```

 "Группы вершин, где из каждой можно дойти до каждой",
 formal: "Косарайю: DFS с сохранением порядка выхода",
 complexity: "O(V + E)",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-функцию",
 "префикс", "префикса", "суффиксом", "суффикс", "суффиксов",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался в этой позиции и какой кусок",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса $s[0..i]$, который совпадает с",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция", "z-функции", "z-функцию"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начиная с этой позиции, совпадает",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n-1]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{

```

```

 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрыв
 formal:
 "Связный граф: эйлеров цикл все степени чётные;
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе
 ,
 formal:
 " $V - E + F = 2$ для связного планарного графа. Кур
 ,
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [

```

```

 "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам",
 formal:
 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки можно сделать, чтобы посадить пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок обхода вершин произвольный.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
 },

```

```

 demo: "components",
 simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "графа",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 "G = (V, E). Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двустороннее движение).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Решение: сжатие координат с помощью сортировки и бинарного поиска.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова структура"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:

```

```

 formal:
 "A ≤p B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы O(1). Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где
 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — O(N·M). Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на
 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: x = p.left; p.left = x.

```



```
 },
];

 /** Быстрый доступ по id. */
 export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

 /** Все метки, отсортированные от длинных к коротким (ч
 export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);
}
```

---

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 275 | РАЗМЕР: 1.2 КБ

---

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6">
 Обзор
 <h2 class="text-3xl font-bold text-white">Обзор</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Кратчайшие пути</h3>
 <p class="text-slate-300 text-sm mb-4">Предметом данной главы является поиск кратчайшего пути (или маршрута) в графе. Алгоритмы поиска кратчайшего пути по весу рёбер (или стоимости проезда). Алгоритмы поиска кратчайшего пути по количеству рёбер (или количеству остановок).</p>
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
 </div>
 </section>
 `
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-6">
 охождение улицы), Дейкстра сломается, так как
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь (min-heap).
 g V). Гарантирует корректность только для не-ориентированных графов.
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <div class="flex items-center mb-6">
 Билет 15
 <h2 class="text-3xl font-bold text-white">Форд-Беллман
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-slate-600">
 <h3 class="text-2xl font-bold text-white">Описание задачи
 <p class="text-slate-300">Дана неориентированная граф с n вершинами и m ребрами.
 Каждому ребру присвоен вес (длина). Требуется найти кратчайшие пути от заданной
 стартовой вершины до всех остальных вершин графа.
 </div>
 <div class="bg-slate-700/50 p-6 rounded-xl border-slate-600">
 <h3 class="text-2xl font-bold text-white">Алгоритм
 <p class="text-slate-300">Алгоритм Форда-Беллмана (Bellman-Ford) — это алгоритм нахождения
 кратчайших путей от одной вершины до всех остальных вершин в графе с
 произвольными весами ребер. Он работает за время O(n * m), что делает его
 эффективным для графов с большим количеством ребер, но медленным для
 разреженных графов.
 </div>
 <div class="bg-slate-700/50 p-6 rounded-xl border-slate-600">
 <h3 class="text-2xl font-bold text-white">Пример
 <p class="text-slate-300">Представим граф с 5 вершинами (A, B, C, D, E) и
 следующими ребрами и весами:

 A-B: 10A-C: 3B-D: 6C-D: 2D-E: 4От A до A: 0От A до B: 10От A до C: 3От A до D: 5 (через C)От A до E: 9 (через D)

```

```

type: "html",
category: "Графы",
content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd" class="
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>
 </div>
</div>
`

```

```
<p class="text-slate-300 text-sm mb-4">Представим, что мы двигаемся по веткам. Если нужного продолжения ("нити страховки") в самый длинный из известных нам не нашли, то мы можем продолжить поиск с самого начала.
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-700">
 <div class="absolute -top-3 left-4 bg-rose-900 p-2 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Представим, что мы нашли и "he", потому что оно спрятано внутри терминальные ссылки (term_link) – прямые модификаторы
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700">
 <summary class="p-4 font-bold text-slate-400 outline-slate-700">
 Внутренняя структура автомата и код BFS (Скрыть)
 </summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>transitions</code>
 <p>ссылку <code>term_link</code> (ближайшая терминальная ссылка)
 <div class="bg-black/50 p-4 rounded-lg font-mono">
```

```
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по префиксу
 int term_link = 0; // Терминальная ссылка: мат. слово
 bool is_terminal = false;
 vector<string> words;
};
```

```
// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
```

```

 <h2 class="text-3xl font-bold text-white">Алгори
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 вы от 0 до N, и уже по ним строим графы.</p>
 </div>
 </div>
</section>`
 }
};
```

---

ФАЙЛ 23/87: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМ

---

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 }
],
```

```

{
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа в v,
 поиска мостов и точек сочленения."
},
{
 question: "Какова временная сложность алгоритма поиска мостов?",
 options: [
 " $O(V^2)$ — ищем мосты в квадрате",
 " $O(V + E)$ — каждый элемент один раз",
 " $O(2^V)$ — полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро."
},
"planarity-euler-formula": [
 {
 question: "Планарный граф имеет V=6, E=12. Возможно ли?",
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ — на грани простоя.
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=6?",
 options: [
 "3 грани",

```

```
const CHANNEL = "algo:viz-state";

export interface VizStateEventDetail {
 chapterId: string;
 /** Строка и функция реально выполненного пользователем */
 line?: number;
 func?: string;
 /** Машинно-читаемые globals + locals: числа остаются */
 variables?: Record<string, DebugValue>;
 /** Имена, изменённые выделенной строкой. */
 changed?: string[];
}

/** Последний снимок хранится, чтобы поздно смонтировать
const latestByChapter = new Map<string, VizStateEventDetail>();

/** Транслировать реальные переменные визуализациям текста
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...snapshot };
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-state", { detail }));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-state", { detail }));
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | null {
 const chapterId = useContext(VizChapterContext);
```

```

 if (!detail || detail.chapterId !== chapterId) re
 // Без явного селектора компонент либо рисует use
 // либо остаётся ручным. Номер Python-строки нико
 const requested =
 selectByVariables && detail.variables ? selectB
 // null означает: компонент рисует runtime-перемен
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(requested))
 const target = Math.max(0, Math.min(Math.max(0, maxStep), requested));
 if (target !== currentStep) goToStep(target);
};

 apply(latestByChapter.get(chapterId));
 const handler = (e: Event) => apply((e as CustomEvent) as any);
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
}, [chapterId, currentStep, maxStep, goToStep, selectByVariables]);
}

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) =>
 typeof value === "number" && Number.isFinite(value) ? value : null;

export const vizString = (value: DebugValue | undefined) =>
 typeof value === "string" ? value : null;

export const vizArray = (value: DebugValue | undefined) =>
 Array.isArray(value) ? value : null;

export const vizRecord = (value: DebugValue | undefined) =>
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;

/* — Выбор демонстрации внутри страницы (вкладки ID / ID)

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

```



```

* Захардкоженная «карта синхронизации» компилятора с ви
*
* Для каждой страницы (главы) здесь вручную перечислен
* которые понимает её визуализация: i, j, k, v, dist, l
* Компилятор выполняет любой Python-код и передаёт реа
* имён напрямую; привязки к номерам или тексту строк э
*/
```

```
export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k,
 name: string;
 /** Роль в алгоритме (показывается во всплывающей под
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}
```

```
export interface PageSync {
 /** Заголовок демонстрации на странице (если интеракт
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тул
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет
 variables: SyncVariable[];
 /** Полный референсный Python-код, который кнопка-гла
 code?: string;
}
```

```
/** Справочные значения, указанные внутри самих симулято
```

```
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем tr
 code: `n = 8` # длина массива
```

```
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]
```

```
v, l, r = 1, 0, n - 1 # вершина и её отрезок
```

```

print("zig-zag: ", x, "↗", g)` ,
 variables: [
 { name: "x", role: "узел, который поднимаем к корню"},
 { name: "p", role: "родитель узла x перед поворотом"},
 { name: "g", role: "дедушка узла x – нужен в случае поворота"},
],
},
"sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j])$ ",
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j])
 variables: [
 { name: "n", role: "длина массива / сторона квадрата"},
 { name: "k", role: "уровень таблицы: ячейка хранит минимум из k предыдущих"},
 { name: "i", role: "начало блока в строке уровня"},
 { name: "j", role: "номер уровня при построении (1-based)"},
 { name: "r, c", role: "строка и столбец ячейки в таблице"},
 { name: "kx, ky", role: "уровни по вертикали и горизонтали"},
],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = заполнение очередного $P[i] = P[i - 1] + a[i]$ ",
 code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
 P.append(P[i - 1] + a[i - 1])

```

```

 .",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)`,
 variables: [
 { name: "top", role: "вершина стека — то, откуда"},
 { name: "v", role: "текущая вершина графа, которую"},
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
 variables: [

```

```

],
},
"graph-components": {
 // отдельного виджета нет, тема опирается на DFS/BFS
 variables: [],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она д
 code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2],
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v) # выход из v!

for v in adj:
 if v not in used:
 dfs(v)
print(order[::-1]) # разворот — ответ`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = вершина из order (в обратном поряд
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:

```

```

vizTitle: "Мосты: DFS + tin/low",
stepNote: "Шаг = строка псевдокода слева; ребро (v,
code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверя
 { name: "tin[v]", role: "время входа в вершину –
 { name: "low[v]", role: "минимум tin, куда можно
 { name: "timer", role: "глобальный счётчик времени
],
},
"euler-path-vs-cycle": {
 vizTitle: "Эйлеров путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине мар
 code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
 if deg[v] % 2:
 odd.append(v)
print(deg, "нечётных:", len(odd))
0 → цикл • 2 → путь • иначе – нельзя`,
variables: [
 { name: "path", role: "текущий маршрут – последов
 { name: "last", role: "последняя вершина пути – о
 { name: "deg(v)", role: "степень вершины: чётная
],
},
"planarity-euler-formula": {

```

```

print("итог:", dist)\,
 variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние",
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) внутри",
 code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
 u, v, w = "S", "A", 5 # шаг: очередное ребро
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
 if i == 1:
 print(f"итерация {i}: {dist}")
 break`,
 variables: [
 { name: "n", role: "число вершин графа – ровно n-1",
 { name: "m", role: "число рёбер, по которым проходят",
 { name: "i", role: "номер итерации (после i итераций)",
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
 { name: "dist[v]", role: "расстояние; если обновилось",
],
},
floyd: {
 vizTitle: "Флойд–Уоршелл",
 stepNote: "Шаг = инициализация матрицы, смена промежуточного",
 code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):

```

```

edges.sort() # по весу

parent = {} # DSU: корень вершины
for v in "ABC":
 parent[v] = v
def find(x):
 while parent[x] != x:
 x = parent[x]
 return x

mst = []
for w, u, v in edges: # шаг: очередное ребро
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print(mst)\,
 variables: [
 { name: "n", role: "число вершин; в остов войдёт r",
 { name: "m", role: "число рёбер, сортируем по вес",
 { name: "i", role: "ребро сортируем по весу w; ра",
 { name: "parent[v]", role: "DSU: кто представител",
 { name: "rank[v]", role: "высота дерева DSU – по k",
],
},
"mst-prima": {
 vizTitle: "Прим",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

start = "A"
heap = [(0, start, "start")] # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start} # дерево растёт от start
print(f"берём {v} за {w} (из {parent})")\,
 variables: [
 { name: "v", role: "вершина, которую только что д",
 { name: "u", role: "ещё не взятый сосед v – канди
```

```

print(pi)`,
 variables: [
 { name: "n", role: "длина строки s, для которой с",
 { name: "i", role: "индекс текущего символа – счи",
 { name: "j", role: "длина текущего наилучшего сов",
 { name: "π[i]", role: "префикс-функция: длина наи",
 ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l",
 code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения p
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`,
 variables: [
 { name: "n", role: "длина строки s", range: "n = ",
 { name: "i", role: "позиция, для которой считаем",
 { name: "l, r", role: "правый крайний Z-блок: отр",
 { name: "z[i]", role: "длина наибольшего общего п",
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем

```



```

for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
 (r, c, k - 1, k - 1) + half, c + half, k - 1, k - 1]))`
 variables: [
 { name: "r, c", role: "левый верхний угол блока, "},
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг 2^k)"},
 { name: "st2[(r,c,k,k)]", role: "минимум квадратной таблицы"},
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери",
 code: `# st2[(r, c, kx, ky)] из вкладки «построение»
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({kx}, {ky})")
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваемого прямоугольника"},
 { name: "kx, ky", role: "уровень самого крупного блока"},
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи",
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3], [10, 11, 12, 13]]
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):

```

```

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * заменяет редактор полным референсным кодом.
 */
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while|
 keep.push(line);
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim() === "")
 keep.pop();
 return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Корпоративная пре
</div>
<p class="text-slate-300 text-sm mb
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сотр
женное обновление).</p>
</div>

<div class="bg-slate-900 p-6 rounded-lg
<div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Матрешки-коробки
</div>
<p class="text-slate-300 text-sm mb
ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
</div>
</div>

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Строгое доказательство / Код (Скр
</summary>
<div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounde
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}</pre>
</div>

```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-2">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

```



```

 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключо
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоить
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">14. Графы. Поиск кратчайшего пути. Дейкстра
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Представьте себе человека, который хочет добраться из пункта А в пункт Б. Он знает, что для этого нужно пройти через пункт В. Но он не знает, сколько времени займет дорога. Он решает, что лучше всего будет, если он будет двигаться в направлении В, пока не достигнет пункта В.
 <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
 </div>
`

```

```

 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-white">16.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-500">
 <h3 class="text-xl font-bold text-indigo-400">Алгоритм Флойда-Уоршелла
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Идея алгоритма
 <p class="text-xl font-mono text-white">
 ...
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-700/50
 border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг class="bg-slate-900 text-pink-400"
 шу себе проходить через вершину k?»
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только

```



```

 ает очень медленно. Мы хотим сделать Джон
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 sm:text-4xl">
 Добавляем фиктивную вершину
 Запускаем от неё медленную анимацию
 «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое р
 Магия! Все веса тепер
 ренные потенциалы сокращаются).
 Теперь смело запускаем быструю анимацию

 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 18
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 </div>
 </div>
</section>`
 }
]

```



```

<div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb
 Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать заголо
</code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Пример вычисления (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcbcd</p>
 <ul class="list-disc pl-5 space-y-2

```

```

type: "html",
category: "Строки",
content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «приклеива
 я начну читать строку отсюда, насколько длинная
 Это самый быстрый способ найти индекс
 >, считаешь Z-функцию, и там, где <code>Z[i]</code>
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>

```

```

 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

 </div>

```

```

 </details>

```

```

 </div>

```

```

</div>

```

```

</section>`

```

```

 },

```

```

 {

```

```

 id: "aho-corasick",

```

```

 title: "23. Алгоритм Ахо-Корасик",

```

```

 type: "html",

```

```

 category: "Строки",

```

```

 content: `

```

```

<section id="aho-corasick" class="mb-12 scroll-mt-10">

```

```

 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```



```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

 </div>

```

```

<div class="space-y-8">

```

```

 <div class="bg-slate-700/50 p-6 rounded-xl border-l">

```

```

 <h3 class="text-xl font-bold text-blue-400 mb-4">

```

```

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">

```

```

 <p class="text-sm text-blue-300 italic mb-2">Осн

```

```

 <p class="text-lg font-mono text-white">Бор (Тр

```

```

 </div>

```

```

 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>

```

```

 </div>

```

```

<!-- Аналогии -->

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```

 <div class="bg-slate-800 p-6 rounded-lg border bo

```

```

 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">

```

```

 Аналогия 1: Паутина (Бор)

```

```

 </details>
 </div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 (V + E).</p>
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм
 </p>
 </div>
 </div>

```

```

<pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}
</pre>

```

```

<pre class="bg-slate-950 p-3 rounded">
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
</pre>

```

</div>

</details>

</div>

</div>

</section>`,

},

{

id: "graph-planar-colors",

title: "7. Графы. Планарные. Покраска",

type: "html",

description: "Формула Эйлера и теорема о 4 красках.",

category: "Графы. Теория",



```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```

```

bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

 </div>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жару",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 10. Графы. Компоненты сильной связности (SCC)
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Продвинутые</h2>
 </div>

```

```

 border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb">
 SCC (Билет 10) склеивает
 </br>

 Точки сочленения (Билет 12)
 ость монолита. Вырвешь точку сочле
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-
 -b border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bol">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе

 Разворачиваем этот список (то есть делаем

 Переворачиваем все ребра в графе

 Идем по <code>order</code>:

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф"
```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold text">
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
 </div>
```

```
 <p class="text-slate-300 text-sm">
 Важно: Это работает то
еются к точкам сочленения. То есть,
ег - это тупик со степенью 1).
 </p>
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-800
rder-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соед
её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
```



ФАЙЛ 30/87: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re

interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
```

```

const currentTin = Array(nodes.length).fill(-1);
const currentLow = Array(nodes.length).fill(-1);

const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${
 v,
 });
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curren

 recordStep(

```

```

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}

```



```

 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "1"}
 />
);
 }}}

```

```

{ /* Nodes */
nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

```

```

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";

```

```

 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";

```

```

 const r = isAp ? 24 : 20;

```

```

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}

```

```

 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая

```

```
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 31/87: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
];
```

```

 { line: 5, text: ' if dist[u] + weight <
 { line: 6, text: ' dist[v] = dist[u]
 { line: 7, text: ' for (u, v, weight) in E: // П
 { line: 8, text: ' if dist[u] + weight < dis
 { line: 9, text: ' return "ОБНАРУЖЕН ОТП
 { line: 10, text: ' return dist' },
};

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расст
 activeCodeLine: 2,
 });

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

```

```

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v
 distances: { ...dist }, previous: { ...prev }
 });

```

```

 });
 }
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}, вес ${liveW}, расстояние ${liveDist[liveU]} -> ${liveDist[liveV]}`
 }
 : baseStep;

```

```

{currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
 </div>
)}

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u)
 const vNode = nodes.find((n) => n.id === v)
 const isChecking = currentStep.checkingEdges
 v;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D'
 && edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'
 else if (currentStep.hasNegativeCycle &&

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
 asNegativeCycle && isCyclePart) ? 4 : 2} ma
 g ? '4,4' : 'none' />
 <circle cx={(uNode.x + vNode.x) / 2}

```

```

 -colors">
 <span className="font-bold text-white"
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === idx
 return (
 <span key={idx} className={`px-2 py-1
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-slate-800/40 text-slate-300'}`
 {edge.v} ({edge.w})

)
 })}
 </div>
)}
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10 p-4"
 <div>
 <h4 className="text-lg font-bold text-rose-300">Таблица расстояний
 <div className="grid grid-cols-4 gap-2 text-slate-300"
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge === n.id
 return (
 <div key={n.id} className={`p-2 rounded
 : 'border-slate-700/60 bg-slate-800/40'}`}>
 <div className="text-xs text-slate-400">Имя узла
 <div className={`font-mono font-bold text-rose-300`}
 {d}
 </div>
 </div>
)
 })}
 </div>
)}
 </div>
</div>
</div>

```

```
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."
 });
```

```
 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
```



```

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
 });
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
 }
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

```



```

 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-slate-400"
 >
 {n.label}
 </text>
 </g>
);
 }}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4 font-mono font-size tracking-wider">
 bfs(graph, start)
 var {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visit' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty()' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]' },
 { line: 7, code: ' if neighbor not in visited:' },
 { line: 8, code: ' visited.add(neighbor)' },
 { line: 9, code: ' queue.push(neighbor)' },
]}.map((cl) => {
 const isActive = step.activeLine === cl.line
 return (
 <div key={cl.line} className={`px-2 py-2 ${

```

ФАЙЛ 33/87: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
```



```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
 }, [cancelHide]);

 useEffect(() => {
 setAnchor(null);
 setSaving("idle");
 }, [chapter.id]);
```

```

 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
 };

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border-emerald-500
 300 hover:text-white hover:border-emerald-500" />
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF средствами браузера"
 className="flex items-center gap-1.5 text-x-small border-indigo-500
 300 hover:text-white hover:border-indigo-500" />
 </button>
 </div>
 </article>
 </>
);

```

```

 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-500">
 <VizChapterContext.Provider value={chapterContext}>
 <viz.Component />
 </VizChapterContext.Provider>
 </section>
)}

 <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
/>
</>
);
};

```



```

 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,

```

```

{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:2,4:3,5:4}
}

```

```

const rawTo = vizString(vars.to) ?? (vizNumber(vars
const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.l
 return byLabel ?? (Number.isFinite(Number(value))
});
const v = toId(rawV);
const to = toId(rawTo);
if (v === null) return null;
const candidates = DFS_STEPS
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => step.currentNode === v && (
 return candidates.at(-1)?.index ?? null;
));
const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === 0}

```

```

 });
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u
 return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
 }}}
 {step.currentNode !== null && step.currentN
 const n = GRAPH_NODES.find(x => x.id ===
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} f
 >;
 })()}
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b
 stroke={cur ? "#fff" : st ? "#34d399"
 strokeWidth={2.5}
 className="transition-all duration-300
 <text x={n.x} y={n.y+3} textAnchor="mid
label}</text>
 {vis && (
 <
 <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor:
 } l:{step.low[n.id]||"?"}</text>
 </>
)}
 </g>;
 })}
 })}

```

```

 <div className="flex flex-col-reverse gap-1"
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-500"
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emerald-500"
 : "bg-slate-900 text-slate-400"
 }`} >Вершина {GRAPH_NODES.find(n=>n.id === id).name}
 <span className="text-slate-500 m-1"

 </div>
))
 }
 </div>
 </div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border border-slate-800"
 <p className="text-[11px] text-slate-300 leading-5"
 <div className="mt-2 pt-2 border-t border-slate-800"
 Мосты:
 <span className="font-mono font-bold text-slate-300"
 {step.bridges.length > 0 ? step.bridges.map(b => b.name) : ""}

 </div>
 </div>
 </div>
</div>
);
}

```

```

];

 const adjList: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });

 const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем минимальный элемент' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbours(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие пути найдены' },
];

 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пикет-Парка',
 activeCodeLine: 2,
 });
 }, []);

```

```

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: `    Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]}!`,
 activeCodeLine: 9,
 });
 }
 }
 }

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: `  Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!`,
 activeCodeLine: 11,
 });

 setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">Закончено!</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);

```





```

 </text>
 </g>
);
 }}}

 { /* Вершины */
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited[node.id]
 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#a5b4fc' : '#f8d7da'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#a5b4fc' : '#f8d7da'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="black">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {node.name}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white">
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 })}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">

```

```

 }}}
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 Таблица расстояний
 </h4>
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. шаг)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-b border-rose-950 ${
 isCurr ? 'bg-indigo-950/60 font-medium' : ''
 }`}>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev ? nodes.find((p) => p.id === prev).name : ''}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
 </div>
 </div>

```

ФАЙЛ 38/87: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';
```

```
interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}
```

```
const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];
```

```
export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(false);
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');
```

```
 useEffect(() => {
 const generatedSteps: DPStep[] = [];
```

```

 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibMemo(${n - 1})`,
 codeLine: 4,
 memoState: { ...memoMap }
 });

 const res1 = simulateFib(n - 1);
 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo(${targetN}) = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);

```

```

<div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 border">
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text">
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
</div>

<div className="flex flex-wrap items-center gap">
 <div className="flex items-center gap-2 bg-sl">
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.targ
 disabled={isPlaying}
 className="bg-transparent text-white font
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950">
 <button
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-w
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set

```

```

 return (
 <div
 key={num}
 className={`flex flex-col items-center
 isCurrent
 ? 'bg-emerald-950 border-emerald-500
 : isCached
 ? 'bg-slate-900 border-emerald-500/50
 : 'bg-slate-900/50 border-slate-800
 `}
 >

 <span className={`text-xl md:text-2xl font-medium
 {isCached ? val : '0'}
 `}>

 {num <= 2 &&
 </div>
 </div>
);
 }
 }
 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-800">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-900 border-slate-800">
 Текущий N: <strong className="text-emerald-400">
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-800">
 <button
 onClick={() => setActiveTab('table')}
 >

```

```

 </div>
 }}

 {activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border-2 border-slate-800" >
 <div className="bg-slate-900 px-6 py-3 border-2 border-slate-800" >

 </div>
 <pre className="p-6 font-mono text-sm space-x-1" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 border-l-4
 ${isActive ? 'bg-emerald-600/20 border-l-emerald-400' : 'bg-slate-900/50 border-l-slate-800'}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-2 border-slate-800" >

 {currentStep?.desc}
 </div>
 </div>
)}
 </div>
)}
 </div>

 { /* Progress Bar */ }
 <div className="mt-8 pt-6 border-t border-slate-800" >

```

```

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, v
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u ===
 const endDegree = C1_EDGES.filter(e => e.u === vId
 const isCycle = startDegree % 2 === 0 && endDegree

```



```

 return <g key={n.id} className="cursor-point
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
 strokeWidth={2.5}
 className="transition-all duration-200"
 <text x={n.x} y={n.y+4} textAnchor="middle">{n.l
 bel}</text>
 </g>;
 }}}}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500 text-white p-2
 Pebër: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-500"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-500"
 : "bg-slate-900/50 border-slate-700 text-slate-500"}>
 {c1Msg}
 </div>
 </div>
);
}

```

ФАЙЛ 40/87: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

```

```
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки',
 options: [
 'Он использует меньше памяти благодаря коммунистиче',
 'Он умеет работать с отрицательными весами без ци',
 'Он позволяет выполнять шаги слияния колхозов (ко',
 'Он был разработан в Токио и поэтому самый быстрый'
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая ком
 лелить вычисления на GPU или многопроцессорных кл
},
{
 id: 4,
 question: 'В чем главная суть Динамического програм
 options: [
 'Писать код очень быстро, динамично нажимая на кл',
 'Запоминать (кешировать) результаты уже решенных',
 'Использовать динамическую типизацию как в Python',
 'Постоянно менять требования к проекту в процессе'
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци
},
{
 id: 5,
 question: 'Какая структура данных идеально помогает
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin',
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 .
}
```

```

 if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-
 my-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-
 -6 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-
 <p className="text-slate-400 text-sm mb-6">Ты пр

 <div className="bg-slate-950 border border-slate-
 <p className="text-slate-400 text-sm uppercase
 <p className="text-5xl font-black text-indigo
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскра
 : score >= 3
 ? ' Отличный результат! Главные концепции
 : ' Стоит повторить главы пособия и еще р
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4
 d rounded-xl shadow-lg shadow-indigo-600/30
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
 }

 return (
 <div className="bg-slate-900/90 border border-slate
 >

```

```

 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-indigo-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-indigo-500" />
) : (
 <div className={"w-6 h-6 rounded-full border border-indigo-500 bg-indigo-500 text-white"}>
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-base">
 {option}
 </div>
 </div>
);
 }}}}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t border-slate-800">
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 >

```

```

 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

```

```

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

```

```

const adjList: { [key: number]: { v: number; w: number }[] } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

```

```

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

```

```

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ' Все пары отработаны. Алгоритм Флойда завершён.'
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const rawMatrix = vizArray(vars?.d) ?? vizArray(vars?.matrix);
const liveMatrix = rawMatrix
 ? baseStep.matrix.map((row, i) => {
 const values = vizArray(rawMatrix[i]);
 return row.map((_, j) => vizNumber(values?.[j]))
 })
 : undefined;

const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;
const currentStep: Step = compilerLinked ? baseStep : null;

```

```

<div className="absolute top-3 left-3 bg-indigo-500 text-white font-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ? "Step " + currentStep.k : ""}
</div>

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-fw-normal" markerWidth="10" markerHeight="10" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth="10" markerHeight="10" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6" markerHeight="6" fill="#818cf8" />
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id === u);
 const vNode = nodesSvg.find((n) => n.id === edge.v);
 const isTarget = currentStep.i === u && currentStep.k === edge.k;
 const isVia = (currentStep.i === u && currentStep.k === edge.k + 1);
 let strokeColor = '#475569'; let marker = 'arrow-fw-normal';
 if (isTarget) { strokeColor = '#10b981'; marker = 'arrow-fw-active'; }
 else if (isVia) { strokeColor = '#818cf8'; marker = 'arrow-fw-via'; }
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
 markerEnd={marker} strokeDasharray={isTarget ? [5, 5] : []}
 strokeColor={strokeColor} strokeWidth={2} />
 <circle cx={({uNode.x + vNode.x) / 2} cy={({uNode.y + vNode.y) / 2}
 fill={isVia ? '#818cf8' : '#64748b'} strokeWidth={1} />
 <text x={({uNode.x + vNode.x) / 2} y={({uNode.y + vNode.y) / 2}
 text={edge.k + 1} fontColor={isTarget ? 'white' : 'black'}
 fontSize="10" fontWeight="bold" />
 </g>
);
 });
 })}
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 })}

```





```
 });
 }
}
```

---

ФАЙЛ 42/87: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
};
```

```

}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начать с узла ${start}`, visitedNodes: [...Array.from(vis)] });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}`, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посетить узел ${u}`, visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Завершить поиск`, visitedNodes: [...Array.from(vis)] });

 return steps;
}

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);

```

```

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row gap-4">
 <div /* SVG Graph View */
 <div className="bg-slate-900 border border-slate-800 p-4
 flex min-h-[300px]">
 <svg className="w-full h-full max-w-[1000px] max-h-[300px]">
 <!-- Edges -->
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);
 // Check if edge is part of current step
 const edgeTraversed = isVisited(u.id) & isVisited(v.id);

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={edgeTraversed ? 'stroke-emerald-500' : 'stroke-slate-500'}
 />
);
 })}
 </svg>
 </div>
 </div>
 </div>
 </div>
);

```

```

)
 }}}}
 </svg>
 </div>

 {/* Structure View (Stack / Queue) */}
 <div className="bg-slate-900 border border-
r">
 <div className={`absolute -top-3 left-
 ${mode === 'dfs' ? 'border-
ld-300 border border-emerald-500'}`}>
 {mode === 'dfs' ? 'Стек вызовов' : ''}
 </div>

 <div className="flex flex-col gap-2 w-
 {step.queueOrStack && step.queueOrStack.length > 0}
 <div key={idx} className={`w-
 ${mode === 'dfs' ? 'bg-slate-900' : 'bg-white'}
text-emerald-200'}`>
 {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `Узел: {item}`
 : ''}

 </div>
 </div>
 </div>

 {(!step.queueOrStack || step.queueOrStack.length === 0) ? '' : ''}
 <div className="text-slate-500" style={mode === 'dfs' ? {} : {color: 'red'}}>
 </div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={mode === 'dfs' ? {} : {color: 'red'}}>
 {/* Controls */}
 <div className="flex bg-slate-900 border

```

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) l
```

```

 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🤒' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩹' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animState: 'idle' });
 });
 return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? vizArray([]);
 const liveI = vizNumber(runtime?.variables?.i);
 const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNumber(0);
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : String(index),
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string>('');
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({

```

```

 }, 500);
 }, [busy, heap]));

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`  СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'removed' } : x));
 }, 400);
}, [busy, heap]);

```

```

 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>
 </div>

```



```

 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:cursor-not-allowed text-white font-medium py-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
 </form>

 { /* Забрать самого тяжелого */ }
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium py-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

```

```

{ /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
<div className="grid grid-cols-1 lg:grid-cols-12 gap-4" >

 { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl p-4 min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
 <div className="absolute top-4 left-4 text-[1.2em] font-medium" >
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

 <div className="font-bold text-white">
 <div className="text-emerald-400">
 </div>
 </div>
 </div>
);
 }}}
</div>

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-600">
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-white">
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-between p-4">
 {healingPatient ? (
 <div className="flex flex-col items-center justify-center h-100">
 <div className="relative w-100 h-100">

 {healingPatient.name}

 {healingPatient.status}

 {healingPatient.status}

 </div>
 <div>
 <h4 className="text-white font-black">
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-x-small">
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-4">

 <Heart className="w-4 h-4 text-rose-500"></Heart>
 </div>
 </div>
 </div>
) : (
 <div>
 <h4>Пациент не найден</h4>
 <p>Пожалуйста, проверьте данные.</p>
 </div>
)}
 </div>
</div>

```

Универсальное пособие • полный исходный код

-----  
import React, { createContext, useContext, useEffect, u

/\*\* Общие примитивы для мини-визуализаций подсказок. \*/

export const W = 260;

export const H = 130;

/\*\*

\* Общий множитель скорости. Кадры были слишком короткими

\* заметить, что именно переехало, и анимация читалась

\*/

export const SPEED = 2.1;

/\*\* Длительность переезда узлов. Должна быть заметно ме

export const MOVE\_MS = 800;

export const EASE = "cubic-bezier(.34,.01,.2,1)";

export const MOVE = `all \${MOVE\_MS}ms \${EASE}`;

/\*\* Наведение на карточку ставит анимацию на паузу – мо

export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {

const [step, setStep] = useState(0);

const paused = useContext(DemoPauseContext);

const period = Math.round(ms \* SPEED);

useEffect(() => {

if (steps <= 1 || paused) return;

const t = setInterval(() => setStep((s) => (s + 1) %

return () => clearInterval(t);

}, [steps, period, paused]);

return step;

}

export const C = {

idle: "#334155",

idleStroke: "#475569",

text: "#e2e8f0",

```
 <text x={x} y={y} textAnchor="middle" dominantBas
 {label}
 </text>
)}
 </g>
);
```

```
export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);
```

---

ФАЙЛ 45/87: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../de

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
```



```

const boxW = 46;
const startX = (260 - raw.length * (boxW + 4)) / 2;

return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}

```

```

 <text textAnchor="middle" dominantBaseline="c
 {id}
 </text>
 </g>
 }}}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions=[["р – корень, х под ним", "Один поворот:
 frames=[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
]}
 />
};

```

```

/** Zig-Zig – х и р с одной стороны, крутим сначала вер
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions=[["Линия g → р → х («бамбук»)", "Поворот В
 frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112
]}
 />
};

```

```

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions=[["Зигзаг: g влево, х вправо", "Поворот НИ
 frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112

```

```

 });
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k} />
))}
 </Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["трусы", "штаны"],
 ["носки", "боты"],
 ["трусы", "боты"],
 ["штаны", "боты"]
];
 const order = ["трусы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, { x: n.x, y: n.y }]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."}`
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(v) < placed.indexOf(u) ? "red" : "gray"} />
))}
 </Svg>
);
};

```



```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
 </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C")) ? null : (
 <Edge key={i} a={pos[u]} b={pos[v]} color={cut ? "red" : "black"} />
))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="none" />
) : (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
)
))}
 </Svg>
);
};

```

```

 };
 const edges: [string, string][] = [{"A", "B"}, {"B", "C"}];
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 && scc.includes(v);
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из каждого вершины есть выход"}>
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={inScc(u, v)} />
))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step === 1 ? "#ccc" : "#fff"} />
))}
 </Svg>
);
 };
};

```

ФАЙЛ 47/87: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit";

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине">
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
))}
 </Svg>
);
};

```

```

 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={15}>
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d} />
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f50"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },

```

```

const step = useLoop(links.length + 1, 900);
return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.
)};
)}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}
 </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n

```

```
};
```

```
export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18} />
 <text x={25 + i * 30} y={29} textAnchor="middle" />
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 13} width={26} height={18}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
 })}
 </Svg>
);
};
```

```
export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
 })}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" fontSize={14} />
 </Svg>
);
};
```

```

 <text x={130} y={106} textAnchor="middle" fontSize=
 п[i] — длина самого длинного «префикс = суффикс
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} wid
 fill={idx === step - 1 ? C.active : fil
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} t
 fontSize={10} fontWeight="700" fill="
)}
 </g>
);
 })
)}
 <text x={130} y={122} textAnchor="middle" fontSize=
 каждая клетка считается один раз и переиспользуе
 </text>
 </Svg>
);
};

```

```

 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```

```

 {viz.hint && <p className="text-[11px] text
</div>
<button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
>
 <X className="w-5 h-5" />
</button>
</div>
<div className="overflow-y-auto overscroll-cont
 <Component />
</div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 50/87: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

```

```

export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}

```

```

interface Props {

```



```

window.addEventListener("keydown", onKey);
window.addEventListener("scroll", onClose, true);
return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
};
}, [anchor, onClose]);

```

```

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

```

```

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >
 Подсказка
 </h4>
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>
 </div>
 </div>
)

```

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizStep';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 4, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 5, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

 const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
 };

 const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'A') return 1;
 if (id === 'B') return 2;
 if (id === 'C') return 3;
 }
 return 0;
 };
}
```

```

 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a
 if (e.id === 'AB' && step > 4) return "url(#arr
 if (e.id === 'BC' && step === 5) return "url(#a
 if (e.id === 'BC' && step > 5) return "url(#arr
 if (e.id === 'CA' && step === 6) return "url(#a
 if (e.id === 'CA' && step > 6) return "url(#arr

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;

```

```

 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра.
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес: 0
 Вычисляем: $-2 + 0 - (-2) =$ -2
 </div>
);
 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ -1
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ 1
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы добавили.
 Все новые веса ребер в графе стали отрицательными.
 При этом старые кратчайшие пути остались кратчайшими!
 </div>
);
 default: return "";
}

};

```

```

 const isS = e.u === 'S';
 if (isS && (step === 0 || s

const midX = (u.x + v.x) / 2;
const midY = (u.y + v.y) / 2;

const edgeColorClass = getEdgeColorClass(e);
const textHigh = isEdgeTextHigh(e);
const textEmer = isEdgeTextEmer(e);

return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(e)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${textHigh}${textEmer}${isS ? 'final' : ''}`}
 >{getEdgeContent(e)}
 </text>
 </g>
)
)
)
}

/** Nodes */
nodes.map(n => {
 if (!isNodeVisible(n.id)) return;
 const pot = getPotential(n);

```

```

 {/* Controls */}
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Шаг на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX_STEP
 ? 'bg-emerald-600/5
 : 'bg-indigo-600 ho
 {step === MAX_STEP ? "Готов
 {step !== MAX_STEP && <Skip
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center
 {Array.from({length: MAX_STEP +
 <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
)}}
 </div>
 </div>
 </div>
</div>
);
}

```

---

```

 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map(({step, index}) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== null));
 return candidates.at(-1)?.index ?? null;
});

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (

```

```

 `DFS 1: зашли в вершину ${u}, помечаем посещенн
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче
 ебра.` ,
 null,
 true,
 -1,
);

```



```

);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент,
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {

```

```

 <Pause className="w-4 h-4 ml-1" />
) : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"

```

```

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f469d";
 markerId = "arrow-transposed";
 } else {
 if {
 step.currentNode === edge.u ||
 step.currentNode === edge.v
 } {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }

 // Adjust slightly for bidirectional link
 const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
 const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

 return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 });
}}}

```

```

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =

```

```

 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 })
 </g>
);
}}
</svg>

```

```

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

```

```

<div className="bg-slate-900 border border-sl
 <p className="text-xs text-white leading-re
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-a
 {/* Top-Sort output / Order stack represent
</div>

```

```

 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 53/87: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
```

```
// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal');
 const [vertices, setVertices] = useState<Vertex[]>(initialVertices);
 const [edges, setEdges] = useState<Edge[]>(initialEdges);
 const [stepIndex, setStepIndex] = useState<number>(0);
 const [heapQueue, setHeapQueue] = useState<string[]>([]);
 const [activeCheatTab, setActiveCheatTab] = useState<'kruskal'>('kruskal');

 const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Представим, что мы хотим начать!',
 type: 'info',
 },
]);

 // --- KRUSKAL STEPS ---
 const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
```

```

 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
 },
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный ц
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет! "',
 type: 'warning'
 }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'bl
 setEdges(prev => prev.map(e => e.status === 'incl
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий клан
 type: 'success'
 }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
```

```

 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
 },
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {

```



```

 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]);
},
// Step 22: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: ' Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Суммарный вес: 3 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
},
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8']),
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую вершину. Добавляем ребра в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8).',
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине D. Ребро D-E (3) добавлено к MST. Очередь (Heap) теперь содержит: B-E(6), E-F(7), E-G(8).',
 }, ...prev]));
 },
 // Step 3: Pop B-E (6)
 () => {
 setEdges(prev => prev.map(e => e.id === 'B-E' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает следующее дешевое ребро B-E (6)',
 description: 'Плесень тянется по ребру B-E к вершине B. Ребро B-E (6) добавлено к MST. Очередь (Heap) теперь содержит: E-F(7), E-G(8).',
 }, ...prev]));
 },
 // Step 4: Pop E-F (7)
 () => {
 setEdges(prev => prev.map(e => e.id === 'E-F' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает следующее дешевое ребро E-F (7)',
 description: 'Плесень тянется по ребру E-F к вершине F. Ребро E-F (7) добавлено к MST. Очередь (Heap) теперь содержит: E-G(8).',
 }, ...prev]));
 },
 // Step 5: Pop E-G (8)
 () => {
 setEdges(prev => prev.map(e => e.id === 'E-G' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает следующее дешевое ребро E-G (8)',
 description: 'Плесень тянется по ребру E-G к вершине G. Ребро E-G (8) добавлено к MST. Очередь (Heap) теперь пуста. MST построен!',
 }, ...prev]));
 },
 // Step 6: Final MST
 () => {
 setLogs(prev => [{
 title: 'Шаг 6: Завершение алгоритма Краскала',
 description: 'Алгоритм Краскала завершен. Построен Минимальное Покрыющее Древо (MST) с суммарным весом 51.',
 type: 'success'
 }, ...prev]));
 }
];

```

```

 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A.',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},

```

```

 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]);
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)']

```

```

 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
 },
 // Step 22: Pop H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]));
 },
 // Step 23: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: 'Успех: Плесень захватила весь граф',
 description: 'Все 9 вершин поглощены. Итоговый граф',
 type: 'success'
 }, ...prev]));
 },
],
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)

```

```

 оз {D, E, F, G, H, I}.'',
 type: 'success'
 }, ...prev]));
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы од-
 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
 }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? {
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский',
 description: 'Оба колхоза объединились по мосту',
 type: 'success'
 }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
() => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шага',
 description: 'Все лишние ребра (B-E, E-H, C-F,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }]);
}
```

```

 });
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
};

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
};

```

```

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';

```

```

 onClick={() => handleReset('prim')}}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow
 : 'text-slate-400 hover:text-slate-
)
 }
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-
)
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
</div>

<button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
>
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
</button>

<button
 onClick={handleNextStep}

```





```

 { /* HTML Overlay for Vertices */}
 <div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%",
 className={"absolute -translate-x-1/2 -translate-y-1/2"},
 font-weight: 'bold', text-align: 'center', border: '2px solid black', transition: 'all 0.2s ease-in-out' }
 id={vertex.id}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'prim' && !isPrimStep} &&

 {vertex.id}

 </div>
 </div>
)}
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom-6" style={{
 background: 'white', padding: '10px', border: '1px solid black',
 shadow: '0 0 10px #000' }} >
 <p className={"font-weight: bold; text-align: center; font-size: 1.2em; margin: 0;"} >
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt-2" >
 Общий вес минимального остова: 51. Все вершины соединены
 </p>
 </div>
)}
 </div>

 { /* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl border-1px border-slate-800 p-4" >
 <div className="p-4 bg-slate-900 border-bottom border-slate-800" >
 <h4 className="font-weight: bold; text-slate-200 text-sm margin: 0;" >
 Ход выполнения
 </h4>
 </div>
 </div>

```

```

 {log.type === 'success' && <CheckCircle/>&&
 {log.type === 'warning' && <HelpCircle/>&&
 {log.type === 'error' && <XCircle/>&&
 {log.type === 'info' && <Play/>&& className="text-2xl"}
 <h5 className="font-bold text-sm text-slate-300">
 </div>
 <p className="text-xs text-slate-300">
 </div>
)))
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-800 p-4">
 <div className="flex flex-col sm:flex-row items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400"/>
 <h4 className="text-xl font-bold text-white">
 </div>

```

```

<div className="flex bg-slate-950 p-1 rounded">
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 "==" + 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
 }
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 "==" + 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-slate-300"
 }
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('dijkstra')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 "==" + 'dijkstra' ? "bg-violet-600 text-white" : "bg-slate-800 text-slate-300"
 }
 >
 Дейкстра (Билет 20)
 </button>

```

```

<pre className="text-xs text-emerald-400" style="font-family: monospace;">
 80 flex-1">
`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
</pre>
</div>
</div>
</div>
}}

{activeCheatTab === 'prim' && {
 <div className="grid grid-cols-1 lg:grid-cols-2" style="font-family: monospace;">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-white">Прим</p>
 <p className="text-slate-400 text-xs font-mono">
 mst
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-mono">
 edges
 </p>
 </div>
 </div>
 </div>
}
```

```

 </div>
 </div>
 </div>
 })

 {activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" >
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
 <p className="text-slate-400 text-xs mt-2">
 Введите время в формате HH:MM:SS
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs mt-2">
 Введите количество баллов
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
 <p className="text-slate-400 text-xs mt-2">
 Введите код
 </p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <pre className="text-xs text-emerald-400">
 80 flex-1">
 </pre>
 </div>
 </div>
 </div>
)
}

`def boruvka(n, edges):

```

```

 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: ')). Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

```

```

 index: number;
 total: number;
 }

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : ""
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escap
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, []);

 return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40"
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2
 aria-label="Открыть содержание"
 >
 <span className="p-1.5 rounded-lg bg-indigo-600"
 <List className="w-4 h-4" />

 <span className="block text-[10px] uppercase"
 Содержание • {index + 1} из {total}

```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Code } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
```

```
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 w-6 h-6 rounded-sm flex items-center justify-center">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
```

```

 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 страницей"
 className={`flex-1 lg:flex-none flex items-stretch
 duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.pdf`}
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center border border-emerald-600/20
 over:bg-emerald-600/20 border border-emerald-600/20"
 title="Скачать полный PDF сборник всех страниц"
>
 <FileDown className="w-4 h-4" /> Скачать PDF

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`}
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center border border-sky-500/30
 :bg-sky-600/20 border border-sky-500/30 transition"
 title="Скачать полный TXT файл со всем кодом"
>
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"

```



```

 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
 }
 const labels = ["A","B","C","D","E"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=

```

```

const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
}
const labels = ["\u04141","\u04142","\u04143","\u04144"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950/40" z-10">
 ▲ Запомни: K5 и K3,3 – два кита непланарности.
 гомеоморфный K5 или K3,3 – он непланарен. Теорема Кюри.
</div>
</div>
);
}

```

```

 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/index.html`;
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed";
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as const;

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохраняется */
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;
 /** Перейти к странице пособия (посмотреть визуализацию) */
 onOpenGuide: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```

 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары по...",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "C", "A")]\nedges\n',
 },
],
},
{
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j — как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик — как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"Итерация {i}\")\ni += 1",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i — позиция, x — элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
 code: 'st = []\nst.append("A") # push\nst.pop() # pull',
 },
],
},

```

```

let savedEditor: { code: string; template: string; chapterId: string } = {
 code: "",
 template: "",
 chapterId: ""
};

export function PythonCompiler({ chapterId, chapterTitle, chapterProps }) {
 /** Активная вкладка демонстрации страницы (у sparse-редактора) */
 const [demoId, setDemoId] = useState<string | null>(null);
 const sync = useMemo(() => getPageSync(chapterId, demoId));
 const syncVarBases = useMemo(
 () => {
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\\s*(?:,|\\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
)],
 [sync]
);
 const syncVarSet = useMemo(() => new Set(syncVarBases));
 /** Полный эталон: глаз сразу вставляет его в обычный редактор */
 const template = useMemo(() => buildSyncTemplate(chapterId, demoId));
 /** Дефолт редактора: только инициализация демо; она не зависит от chapterId */
 const initTemplate = useMemo(() => buildInitTemplate());

 const [code, setCode] = useState(() => {
 if (!savedEditor) return initTemplate;
 if (savedEditor.code === savedEditor.template || savedEditor.chapterId !== chapterId)
 return savedEditor.code;
 });

 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Код выполнится");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 /** Принудительный повтор (глаз / Ctrl+Enter), даже если код не готов к выполнению */
 const [traceRequest, setTraceRequest] = useState(0);
 const [copied, setCopied] = useState(false);

```

```

useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId(d)), [chapterId]);
useEffect(() => () => clearVizState(chapterId), [chapterId]);

// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev.full) {
 prevTplRef.current = { base: initTemplate, full: template };
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого кода
 if (code === prev.base || code === prev.full || code === prev.partial) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {}
}, [code]);

```

```

 }
 const changed = changedVars(step?.locals, locals);
 return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => ch
 };
 }, []));

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 }
);

```

```

const source = code;
const input = stdin;
const request = traceRequest;
setRunning(true);
setPlaying(false);
setEditHold(null);
setDebug(null);
clearVizState(chapterId);
setOutputOpen(false);
setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0) ? inputLines.shift() : null });

 const raw = await runtime.runPythonAsync(buildDebugRequest());
 const result = JSON.parse(String(raw)) as DebugRequestResult;

 setDebug({ result, idx: 0, src: source, input, request });
 setPlaying(autoPlay && result.steps.length > 1);
 const printed = [...stdout, ...stderr].join("");
 setOutput(printed || "Программа ничего не вывела.");
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEP_LIMIT} шагов`
 : "Трасса готова автоматически. ← → — шаги, П — пауза");
} catch (error) {
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, truncated: true }, idx: 0, src: source, input, request });
 setOutput(`Ошибка:\n${message}`);
}

```



```

 }
 if (e.key === "Escape") stopDebug();
 };
 // capture-фаза: чтобы глобальный обработчик навига
 window.addEventListener("keydown", onKey, true);
 return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current)
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.sr
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(cu
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {

```

```

 aria-orientation="horizontal"
 aria-label="Потяните, чтобы изменить высоту терминала"
 title="Потяните, чтобы изменить высоту терминала"
 className="absolute -top-1.5 left-0 right-0 h-100"
 onPointerDown={(event) => {
 const el = event.currentTarget;
 el.setPointerCapture(event.pointerId);
 const startY = event.clientY;
 const startH = el.parentElement?.getBoundingClientRect().height;
 const onMove = (moveEvent: PointerEvent) => {
 const maxHeight = window.innerHeight - (window.topBarHeight || 0);
 const minHeight = Math.min(260, maxHeight);
 const next = Math.round(startH + startY - moveEvent.clientY);
 onHeightChange(Math.max(minHeight, Math.min(maxHeight, next)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove);
 el.removeEventListener("pointerup", onUp);
 el.removeEventListener("pointercancel", onUp);
 };
 el.addEventListener("pointermove", onMove);
 el.addEventListener("pointerup", onUp);
 el.addEventListener("pointercancel", onUp);
 })
 }
 >
 <div className="absolute left-1/2 top-1/2 h-100 w-100
 dingo-400 transition-colors" />
 </div>
)}

 {/* Левый край меняет ширину на десктопе. */}
 {onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину панели"
 title="Потяните, чтобы изменить ширину панели"
 className="hidden lg:block absolute top-0 bot

```

```

 className={`inline-flex items-center gap-0 ${
 playing
 ? "bg-amber-500/15 text-amber-300 hover:"
 : "bg-emerald-500/15 text-emerald-300"
 }`}
 aria-label={running ? "Python выполняется" : ""}
 >
 {running ? <Loader2 className="h-4 w-4 animate-spin" /> : ""}
 {playbackSpeed}x
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скопировать код" >
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-emerald-500" /> : <Copy className="w-4 h-4 text-slate-400" />}
 </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть комментарий" >
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель" >
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>

```

```

 >
 {PLAYBACK_SPEEDS.map(({speed}) => (
 <option key={speed} value={speed}>{speed}
))}
 </select>
 </label>
 </div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргалка */}
<div className="shrink-0 border-b border-slate-800" >
 <div className="flex items-center gap-1 px-2 pt-2" >
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-white" : ""
 }` >
 >
 <Variable className="w-3.5 h-3.5" /> Переменные
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text-white" : ""
 }` >
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказки
 </button>
 <div className="ml-auto pr-1" >
 <Tooltip
 side="bottom"
 content={
 <div>
 hasViz
 ? "Связь идёт по именам ниже: реальный код"
 : "На выбранной странице нет интерактивных элементов"
 </div>
 }
 />
 </div>
 </div>
</div>

```

```

 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink
Страница сменилась – подставить её пер
 </button>
 </Tooltip>
)}
{hasViz && {
 <div className="flex flex-wrap items-centr
 {sync.variables.map((v) => {
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
 >
 {v.name}
 </code>
 </Tooltip>
)}}
 {sync.stepNote && {
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-fle

```

```

{ /* — Редактор + ввод
<div className="flex-1 min-h-0 flex flex-col">
 {debug ? (
 <div className="flex-1 min-h-[80px] flex flex-col">
 { /* панель управления шагами */
 <div className="shrink-0 flex items-center justify-center">
 <Tooltip content={playing ? `Пауза автопро
роиграть трассу заново со скоростью ${playbackRate} раз`}>
 <button
 type="button"
 onClick={runOrTogglePlayback}
 className={`mx-0.5 inline-flex items-center justify-center w-500/15 hover:bg-amber-500/25`} : "text-emerald-500"
 aria-label={playing ? `Пауза автопроигрывателя`}>
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}

 {playing ? "Пауза" : "Воспроизведение"}

 </button>
 </Tooltip>
 <Tooltip content="В начало трассы">
 <button type="button" onClick={() => {startFromBeginning()}}>
 <md text-slate-400 hover:text-white hover:bg-slate-500>
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопроигрыватель">
 <button type="button" onClick={() => {previousStep()}}>
 <md text-slate-400 hover:text-white hover:bg-slate-500>
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопроигрыватель">
 <button type="button" onClick={() => {nextStep()}}>
 <md text-slate-400 hover:text-white hover:bg-slate-500>
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 </div>
 </div>
 </div>
 </div>
 </div>
)}

```

```

 <button type="button" aria-label="Редак
p-1 rounded-md text-rose-400 hover:text-wh
 <Square className="w-3.5 h-3.5" />
 <span className="text-[9px] font-bold
 </button>
 </Tooltip>
</div>

```

```

{/* листинг с подсветкой текущей строки */}
<div ref={listRef} className="flex-1 min-h-1
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.l
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-1
 active ? "bg-emerald-500/15 text-em
 }`}
 >
 <span
 className={`w-7 shrink-0 select-n
 active ? "text-emerald-300" : "
 }`}
 >
 {active ? "►" : ln}

 {line || " "}
 </div>
);
 })}
</div>

```

```

{/* панель переменных – «Variables», как в
<div className="shrink-0 max-h-[34%] overfl
 <div className="flex items-center gap-1.5
 <Variable className="w-3 h-3 text-indig

```

```

 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-1 min-h-[90px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-blue-500"
 placeholder="Напишите Python-код..."
 />
))
 <div className="shrink-0 border-t border-slate-500" >
 <label htmlFor="python-stdin" className="text-slate-500">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono text-slate-500"
 />
 </div>
</div>

```



```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍌', color: 'from-blue', bubbleText: 'голода после матана!' },
 { name: 'Кодер Семён', emoji: '🍜', color: 'from-violet', bubbleText: 'ищу ИИ за порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'еда без сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', bubbleText: 'супчик' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', bubbleText: 'салат с оливками и сливками для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate', bubbleText: 'сметана, плиз. тартаны.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'from-blue', bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍜', color: 'from-violet', bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'еда без сметаны, плиз.', animState: 'idle' },
]));

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
```

```

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c), 450);
 }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '🍲' } : c));

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue shrinks
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '😊' } : c));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`- ${headGuest.name} поел и ушел сытым. 🍴`);
 }, 400);
 }, 900);
}, [queue, isServing]);

```

```

 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
 font-mono rounded-2xl px-4 py-3 shadow-lg hover:scale-95 transition-all cursor-pointer"
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
 font-mono rounded-2xl px-4 py-3 shadow-lg hover:scale-95 transition-all cursor-pointer flex
 items-center justify-center"
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12 gap-4" >

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 min-h-[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
 <div className="absolute top-4 left-4 text-white font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ.length}
 {liveQ &&
 {liveQ[0]}
 }
 </div>

```

```

const isHead = idx === 0;
const isTail = idx === displayQueue.length - 1;
return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center gap-2
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -translate-x-50%
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border-1px solid black' : 'opacity-0'}
 `}>
 <div className="font-bold text-slate-200">
 {customer.bubbleText}
 <div className="absolute -bottom-16 left-1/2 -translate-x-50%
 it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-1px solid black
 relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[10px]
 ${isHead ? 'bg-amber-500' : 'bg-slate-200'}

```

```

 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded"
 >
 {item}
 </div>
))}
 </div>
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r from-indigo-500 to-purple-500"
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-400"
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center gap-2`}
 >
 {idx === 0 ? Пол : }
 {entry}
 </div>
))}
 </div>
 </div>
);
};

```

---

```

 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []));

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 }
 });
};

```

```

 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД),
 S, чтобы расставить fail-ссылки!`
);
 };

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове!`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем построить БПД!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const visited = new Set();
 while (queue.length > 0) {
 const node = queue.shift();
 visited.add(node);
 if (node.children.length > 0) {
 queue.push(...node.children);
 }
 }
 return visited;
};

```

```

 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400" >

```



```

/>

{ /* Плавники */
<path d="М 90 55 L 70 30 L 110 55 Z" fill="black" />
<path d="М 90 145 L 70 170 L 110 145 Z" fill="black" />
<path
 d="М 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate"
/>

{ /* Глаз (с морганием) */
<circle cx="145" cy="85" r="12" fill="black" />
{isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" />
) : (
 </>
 <circle cx="148" cy="85" r="6" fill="black" />
 <circle cx="151" cy="82" r="2" fill="black" />
</>
)}

{ /* Рот (анимация разговора) */
{isTalking ? (
 <path d="М 165 105 Q 155 120 165 125 Q 165 130 155 135 Z" />
) : (
 <path d="М 165 110 Q 155 115 168 118" stroke="black" />
)}

{ /* Улыбка / жабры */
<path d="М 130 80 Q 125 100 130 120" stroke="black" />
<path d="М 120 85 Q 115 100 120 115" stroke="black" />
</svg>

{ /* Декоративные пузыри */
<div className="absolute -top-1 right-4 text-white" />
<div className="absolute top-8 right-1 text-white" />
</div>

```

```

 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-700 shadow-blue-900/40 transition-transform"
 >
 Шаг BFS

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-700 shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>
 </div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center" >
 Дерево автомата
 Дерево и Суффиксные (fail) ссылки
 </h5>

```

```

 {{{() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);

```

```

 >
 {node.char}
 </text>
 </g>
);
}}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes).map(node => {
 if (node.id === 0 || node.fail === 0) return null;
 // Wait, all nodes get fail link. Let's draw it at step
 // To avoid too many lines, let's draw it at step
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return null;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node;

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${node.x + 32} ${node.y - 10}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}

{/* Draw Nodes */}
{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node;
 const stroke = isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)';
 const strokeWidth = isCurrent ? 2 : 1.5;
 const fill = isRoot ? 'white' : 'none';
 const strokeDasharray = isRoot ? "4 4" : "0 0";
 const markerEnd = isRoot ? "url(#root-arrow)" : "url(#fail-arrow)";

 return (
 <g>
 <circle
 cx={node.x}
 cy={node.y}
 r={10}
 fill={fill}
 stroke={stroke}
 strokeWidth={strokeWidth}
 strokeDasharray={strokeDasharray}
 markerEnd={markerEnd}
 />
 <text
 x={node.x}
 y={node.y}
 text={node.char}
 font-size={16}
 font-weight="bold"
 />
 </g>
);
}}

```

```

 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
)}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700
 px-2 py-1 rounded text-white font-medium"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-slate-400
 text-sm font-normal"
 title="Удалить"
 >
 x
 </button>
 })}
 </div>
 </div>
 </div>

```

```

<table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800" >
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }`}
 >
 <td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full
 ${node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }
 >

```

```

 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (

 {node.words.join(', ')}

) : (

 {node.words.join(', ')}

)}
 </td>
 </tr>
 </tbody>
 </table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 61/87: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizState';

```

```
// ===== TYPES =====
```

```

interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index to value
}

```

```

 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
 lightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: ${tree[1]}`,
 Highlight: [0, n - 1] });
return steps;
}

// 2. Top-Down Query steps
function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

```

```

let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL}, r = qR + n = ${qR}, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l += 1. Сумма = ${res}. l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l] });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный → r--`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [r] });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = qL, r = qR, codeLine: 7, treeState: tree, highlightNodes: [l, r], arrayHighlight: [qL, qR], result: res` });
 // ... (omitted code) ...
 }
}
steps.push({ id: id++, desc: `Запрос sum([qL..qR])`, codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

interface VNode { v: number; l: number; r: number; val:

function buildVisualTree(treeState: Record<number, number>):



```

 " l >>= 1; r >>= 1;",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }): {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' },
 emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' },
 slate: { btn: 'bg-slate-600 hover:bg-slate-500 border-slate-500 bg-slate-600', childRing: 'ring-slate-900' },
 amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' },
 [color]: { btn: 'bg-' + color + '-600 hover:bg-' + color + '-500 border-' + color + '-500 bg-' + color + '-600', childRing: 'ring-' + color + '-900' } };

 return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arrType }: { title: string; icon: string; steps: Step[]; code: string; color: string; arrType: 'array' | 'object' | 'number' }): {
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed } = p;

```

```

 { /* Header */ }
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-b-2 border-indigo-500`>
 <h4 className="font-bold text-sm text-white flex-1">

 </div>

 { /* Array strip */ }
 <div className="px-3 py-2 bg-slate-950 border-b-2 border-indigo-500">
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.startIndex;
 return (
 <div key={i} className={`flex flex-col items-center justify-between
 go' ? 'bg-indigo-950 border-indigo-500' : 'bg-slate-950 border-indigo-500'
 er-500'} -translate-y-0.5` : 'bg-slate-900 border-indigo-500'`>

 </div>
);
 });
 </div>

 { /* Tree */ }
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950">
 <div className="scale-[0.85] origin-top min-w-max">
 </div>

 { /* Code */ }
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded-sm border-indigo-500 text-indigo-200
 2 border-indigo-500 text-indigo-200' : 'bg-amber-600/20 border-l-2 border-amber-600/20'`>


```

```

 <option value={120}>Турбо</option>
 </select>
 </div>

 {/* Progress bar */}
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {

```

```

 <span key={i} className={`border px-2 py-1
ext-amber-200 ring-2 ring-amber-400/30`} : "liveI

 </div>
 </div>
</form>

```

```

{ /* Query range */ }
<div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
 <label className="text-[10px] font-bold uppercase">
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 Left
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 Right
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 </div>
 </div>
</div>

```

```
{view === 'queries' && {
 <div className="grid grid-cols-1 xl:grid-cols-2">
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`}
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`}
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
})}
```

```
{view === 'theory' && {
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl border">
 <h4 className="text-base font-bold text-white">
 Почему разбиение на N частей?
 </h4>
 <p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-900 px-1">N</code>
 и получает <code className="bg-slate-900 px-1">N</code>
 частей. Каждая часть забирает на 1 больше (округление в
 большую сторону).
 </p>
 <div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L</div>
 </div>
 </div>
 </div>
})}
```





```

import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' '
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' '
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]));
 const [dequeueMessage, setDequeueMessage] = useState<
```



```

 });

 const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
 };

 // Queue handlers
 const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson.name} ${randomPreset.icon}`);
 };

 const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет новых посетителей.");
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (Ф.И.О. ${firstPerson.name}) ${firstPerson.icon}`);
 };

```

```

 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 });

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', },
 { id: '3', text: 'Кандидат: "Привет, я испанский"', },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability > a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-purple-600">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md
 ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500 text-blue-400' : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
 `}
 >
 <div className="flex items-center gap-3">

```

```

>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}

```

```

 <ArrowDown className="w-3.5 h-3.5 ani
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map(({plate, in
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop
 ? 'bg-gradient-to-r from-blue
le-102'
 : 'bg-slate-900/90 border-sla
 }`}
 >
 <div className="flex items-center
 {pla
 <span className={`font-bold tex
 {plate.name}

 </div>
 <span className={`text-xs font-mo
 isTop ? 'bg-blue-500 text-white
 }`}>
 {isTop ? 'Готово к помывке' : `

 </div>
 });
 }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
}}
```

```

 </button>
 </div>
 </div>

 {dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sla
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
) : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">

 <span className="text-xs font-bold font
 </div>

 <div className="text-indigo-500 flex it
 <ArrowRight className="w-6 h-6 animat
 </div>

 {queue.map((person, index) => {
 const isFirst = index === 0;
 return {
 <div
 key={person.id}

```



```

 {heapMessage && {
 <div className="bg-emerald-950/60 border border-1
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rounded
 {heapMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl
 >
 {sortedHeap.length === 0 ? {
 <div className="text-center py-12 text-slate-400
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте гирю
 </div>
) : {
 <div className="w-full max-w-2xl space-y-3
 <div className="flex items-center justify-between
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 100).toFixed(
 00/10 scale-[1.01]')
 : 'bg-slate-900/80 border-slate-700
 }`}
 >

```

```
 <div className="mt-6 text-xs text-slate-500" style={style}>
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>
 </div>
 </div>
)}
</div>
);
};
```

---

ФАЙЛ 64/87: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
 /** Открыть тренажёр в модальном окне. */
 onOpen: (vizId: string) => void;
 /** Перейти к теме, к которой привязан тренажёр. */
 onGoChapter: (chapterId: string) => void;
}
```

```
/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
```

```
export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");
```

```
 const items = useMemo(() => {
```



```

 className="absolute right-2 top-1/2 -translate-x-1/2
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
 })
 </div>

```

```

{items.length === 0 && <p className="text-slate-500">

```

```

<div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 border-slate-800"
 >
 <h3 className="font-bold text-white text-sm">{entry.title}</h3>
 {entry.hint && <p className="text-[12px] text-white">{entry.hint}
 {!entry.hint && <div className="flex-1" />}
 }

 <div className="flex items-center gap-2 mt-2">
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 text-white transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Запустить
 </button>
 {chapterTitle && (
 <button
 onClick={() => onGoChapter(id)}
 title={chapterTitle}
 className="flex items-center gap-1.5 text-white transition-colors min-w-[150px]
 0 hover:text-white transition-colors min-w-[150px]"
 >
 <BookOpen className="w-3.5 h-3.5 shrink-0" />
 {chapterTitle}
 </button>
)}
 </div>
)}

```

```

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
 },
 {
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf
 const n4: TNode = { key: 4, left: leaf(3), right:

```

```

 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/* ----- раскладка -----

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 };

```

```

 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)
const expected = useMemo(() => (solved ? null : canonicalNext(
tree, target));
setHistory((h) => [...h, clone(tree) as TNode]);
setMoves((m) => [...m, { node: key, correct: want?.length
setTree((t) => rotateUp(clone(t) as TNode, key));
});

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target)

return (
 <div className="w-full bg-slate-950 rounded-2xl border
 <div className="flex items-start justify-between
 <div>
 <h4 className="text-sm sm:text-base font-bold
 <p className="text-[12px] text-slate-400 lead
 Клик по узлу = один поворот с его родителем
 <span className="font-mono font-bold text-in
 поворотов выбираешь сам, разбор покажем в к

```

```

 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
 }

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475569"}
 stroke={isTarget ? "#a5b4fc" : "#475569"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
 })}

```

```

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40" >
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

{solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 >
 <div className="flex items-center gap-2 font-mono" >
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`
 }
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

)
)
 }

)}
 </div>
)}
{wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}

```

```

text: string;
/** Пара, которую крутим на этом шаге: кадр-«прицел».
focus?: [NodeId, NodeId];
/** Кто переехал на этом кадре – рисуем призрак и стрелку.
moved?: NodeId[];
ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",

```

```
const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит
фокус: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему по
убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 }
]
}
```



```

keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],

```

```

 </marker>
 </defs>

 { /* призраки: где узел стоял до поворота */
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill="none" />
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
 })}

 {edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line

```

```

 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
 }}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].p;
 const duration = useMemo(() => Math.round(current.ms - prevPos), [current]);
 const last = idx === total - 1;

 useEffect(() => {
 setFrame(0);
 }, [caseIdx]);

 useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);

```

```

 </div>
 <p className="text-[13px] text-slate-400 leading-4">
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map((t, i) => {
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-sm
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : ""
 }`}
 >
 {t}

))}

 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

 {/* индикатор автопрокрутки */}
 <div className="h-1 w-full bg-slate-800 rounded-full">
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 style={
 playing
 ? { animation: `demoProgress ${duration}ms linear 0s 1` }
 : { width: "100%", background: "#334155" }
 }
 </div>
 </div>

```

```

 : "bg-slate-800 border-slate-700 text-slate-500"
 }` }
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.2"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3 text-sm font-medium transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center justify-center"

)
)}
 </div>
</div>

<p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не пропустят
</p>
</div>
);
};

```

```

const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi-balanced tree
 // Values: 10, 20, 30, 40, 50, 60
 const createInitialTree = (): SplayNode => {
 let r: SplayNode | null = null;
 const initialKeys = [40, 20, 50, 10, 30, 60];
 initialKeys.forEach((key) => {
 r = insertBST(r, key);
 });
 return r!;
 };

 const [tree, setTree] = useState<SplayNode>(createInitialTree());
 const [inputValue, setInputValue] = useState<string>('');
 const [log, setLog] = useState<string[]>([
 'Дерево инициализировано. Попробуйте кликнуть на узел.',
]);
 const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

```

```

 } else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем
);
 }
 }

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right
 if (node.left.right) {
 node.left = leftRotate(node.left);
 steps.push(`Компонент Zig-Zag: левый поворо
 }
 }
 }

 if (!node.left) return node;
 steps.push(
 `Финальный поворот Zig (Правое вращение корня
);
 return rightRotate(node);
} else {

```

```
// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree,
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

const resetTree = () => {
 setTree(createInitialTree());
 setHighlightedNode(null);
 setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = (
 node: SplayNode | null,
 x: number,
 y: number,
```



```

 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 data-title={`Кликните, чтобы выполнить Splay(${
 >
 <circle
 cx={node.x}
 cy={node.y}
 r="18"
 fill={isHighlighted ? "#4f46e5" : "#1e293b"}
 stroke={isHighlighted ? "#818cf8" : "#64748b"}
 strokeWidth={isHighlighted ? "3" : "2"}
 className="transition-all duration-300 group-l
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"

```

```
</svg>
```

```
{/* Quick interactive controls */}
```

```
<div className="absolute bottom-4 left-4 right-
```

```
 <form onSubmit={handleFind} className="flex
```

```
 <input
```

```
 type="number"
```

```
 placeholder="Поиск..."
```

```
 value={inputValue}
```

```
 onChange={(e) => setInputValue(e.target
```

```
 className="w-24 bg-slate-950 text-white
```

```
00 focus:outline-none"
```

```
 />
```

```
 <button
```

```
 type="submit"
```

```
 onClick={handleFind}
```

```
 className="bg-indigo-600 hover:bg-indigo-
```

```
 >
```

```
 Поиск / Splay
```

```
</button>
```

```
<button
```

```
 type="button"
```

```
 onClick={(e) => {
```

```
 e.preventDefault();
```

```
 const val = parseInt(inputValue);
```

```
 if (!isNaN(val)) {
```

```
 let newTree = cloneTree(tree);
```

```
 newTree = insertBST(newTree, val);
```

```
 const { newRoot, steps } = splaySte
```

```
 if (newRoot) setTree(newRoot);
```

```
 setLog([`Вставили [${val}].`, ...st
```

```
 setInputValue("");
```

```
 setHighlightedNode(val);
```

```
 }
```

```
 }}
```

```
 className="bg-indigo-950 hover:bg-slate
```

```
0"
```

```
>
```

```

 </p>
 <p>

 оба левые или оба правые. Вращаем деда, поворачиваем
 </p>
 </p>
 <p>

 колено. Вращаем узел в разные стороны.
 </p>
 </div>
 </div>
</div>

{/* Answers Section inside the visualizer widget */}
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-slate-300">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нытья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 <div className="bg-slate-900/60 p-4 rounded-lg">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>
 <p className="text-slate-400 leading-relaxed">
 Допустим, мы ищем{" "}
 <code className="text-indigo-400 font-sans">{key}</code>
 его нет. Алгоритм спустится к листу, на котором
 неудачей (например, {" "})
 <code className="text-indigo-400">[20]</code>
 <code className="text-indigo-400">[30]</code>
 поднимет в корень{" "}
 <strong className="text-white">
 последний успешно просмотренный узел

 , на котором он споткнулся! Это оставляе

```

```

 всю структуру вдвое более плоской за счет
 при двойных вращениях {
 <code className="text-emerald-400 font-
 }. Дорогая операция инвестирует в дешевые
 операций. Поэтому амортизированное время
 <strong className="text-white"> $O(\log n)$
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emerald-400">
 Каждое "растяжение" компенсируется "сжатием"
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 68/87: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE PRESETS = [

```

```

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);

```

```

 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

const topIndex = displayStack.length - 1;

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}>

```

```

 <div className="absolute w-4 h-4 rounded-
"></div>
 <div className="absolute w-5 h-5 rounded-
"></div>
 <div className="absolute text-4xl anim-cr
</div>
)}

{displayStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {displayStack.slice().reverse().map(({plate
 const idx = displayStack.length - 1 - r
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`

```

```

 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-2 w-full
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
 items-center justify-center gap-2 opacity-1
 >
 {plate.n
 <span className="text-[10px] text-e
 </div>
)}}
 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r f
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-:
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-blue-500
 {entry}
 </div>
)}
 </div>
)}

```



```

 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }
 }
}

```

```

const { s: demoString, steps, patternLength: demoPa
 if (mode === "kmp") return generateKmpSteps(safe
 else return generateZSteps(safePattern, safeText
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

```

```

<div className="bg-slate-800 p-1 flex items-center justify-start">

 <input value={pattern} onChange={onPatternChange} type="text"/>

</div>
<div className="bg-slate-800 p-1 flex items-center justify-start">

 <input value={text} onChange={onTextChange} type="text"/>

</div>
</div>
</div>

<div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-full">
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-400";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(index)) {
 borderColor = "border-emerald-500";
 bgColor = step.match === "match" ? "bg-emerald-900" : "bg-slate-900";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-500";
 }
 }
 })}
 </div>
</div>

```

```

 </div>
 }}
 {mode == "kmp" && step.i
 <div className="abs
 <span classNam
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span classNam
 </div>
 }}

 {/* Z fingers */}
 {mode == "z" && step.i
 <div className="abs
 <span classNam
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span classNam
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span classNam
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ==
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 {step.found && mode ==

```

```

 <div className="text-sm text-slate-500">
 {step.desc}
 </div>
 </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400 leading-relaxed">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "применяется" : "не применяется"}
 о было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
 </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 }
}
`}</pre>

```

```

 className={`relative inline-flex ${className} ?? "
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1
 bg-slate-900 px-3 py-2 text-left text-[11px]
 l duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full
 } ${open ? "visible translate-y-0 opacity-100"
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
};

```

ФАЙЛ 71/87: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,
 Pause,
 RotateCcw,

```

```

 dfsStack: number[];
 }

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла

```

```

 for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
 }

 recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
 setSteps(listSteps);
 setCurrentStepIdx(0);
 }, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

```



```

 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
 </defs>

 {/* Edges */}

```

```

 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"

```

```

 getTopoListString()
 })
</div>
</div>

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Назад
 </button>
 </div>
</div>

```

```

import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeViz";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";

```

```

/** Разбор поворота и песочница всегда идут парой: посмотри
**

```

```

* Анимация и песочница идут друг под другом, а не в две
* колонке дерево сжималось до нечитаемого размера, а по
* компактности. Между ними – подпись, объясняющая перемену
*/

```

```

const SplayRotationsPair: React.FC = () => (
 <div className="space-y-4">
 <SplayRotationsViz />
 <p className="flex items-start gap-2 text-[12px] leading-3 py-2">

 Разобрался – проверь себя: ниже то же самое дерево
 не решишь.

 </p>
 <SplayRotationSandbox />
 </div>
);

```

```

export interface VizEntry {

```

```

"sparse-table": {
 title: "Разрезанная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку – подсветится",
 Component: SparseTableViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число – увидите, из чего оно сложилось",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay-дерево",
 hint: "Покадровый разбор трёх поворотов, песочница",
 Component: () => (
 <div className="space-y-10">
 <SplayRotationsPair />
 <SplayTreeViz />
 </div>
),
},
"splay-rotations": {
 title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
 hint: "Сверху – покадровый разбор поворота, снизу – результат",
 Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDFS },
"top-sort": { title: "Топологическая сортировка", Component: TopSort },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC },
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulation },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: Bridges },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPath },
"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: Planarity },
"dijkstra": {

```

```
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
},
intro: { title: "Мнемокарточки", Component: MnemonicC
"everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез
 Component: Simulator,
},
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

---

ФАЙЛ 73/87: `src/components/WaterfallAnimationWidget.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}
```

```

const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>(
 'Режим поиска: Введите текст и жмите «Следующий шаг»');
const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number, match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

```

```

 let temp = curr;
 while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) => {
 currentMatches.push({ index: currentIndex, word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
 }

 if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog[0].word} `;
 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMatches]);
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {

```



```

 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ ШЕДОВЕЛОМ?',
 desc: 'ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2. Рыба плывет в ROOT и ищет ветку "H". У корня суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель #3 и #4 не подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель #6 не подходит. Но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },

```

```

 </h4>
 <p className="text-sm text-slate-400">
 Визуализация пошагового построения всех f
 </p>
 </div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg sl
 : 'text-slate-400 hover:text-white'
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3

```

```

 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
 ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
 </div>
 </div>

```

/\* Блок с кодом построения fail-ссылки и опи

```

<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail}.label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite

```

```

 » Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
 />
</div>

<div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}`}
 >
 {char}

)}

 </div>
 {currentIndex >= textToScan.length && (

 ГОТОВО ✓

)}

```

```

 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && <span
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && <span
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounded
tems-center">
 {searchLog}
 </div>
</div>
</div>
</div>
))

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
 <div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>

```

```

 />
 <text x="32" y={level.y + 4} fill="#94a8b0">
 {level.label}
 </text>
 </g>
)))

 /* Рендер прямых переходов (потоков воды) */
 Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([charId, charNode]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 type === 'normal';

 // Подсветка в режиме обучения (когда ищем слово)
 let isTrainingExamined = activeMode === 'train' ||
 type === 'normal';
 let isMatchBranch = isTrainingExamined && charNode.char === toNode.char;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 /* Линия течения воды */
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#94a8b0'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 /* Символ перехода */
 <circle cx={({fromNode.x + toNode.x}) / 2} cy={({fromNode.y + toNode.y}) / 2}
 r={10} fill={isMatchBranch ? '#10b981' : '#f43f5e'} />
 <text
 x={({fromNode.x + toNode.x}) / 2}
 y={({fromNode.y + toNode.y}) / 2}
 fill={isMatchBranch ? 'white' : 'black'}
 font-size={12}
 font-weight={isMatchBranch ? 'bold' : 'normal'}
 >{charNode.char}
 </g>
);
 });
 });
}

export default WaterfallDiagram;

```



```

 }}

 { /* Индикатор словарных слов */ }
 { hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
);
}}

{ /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ * }
<g>
 { /* Круг-подложка под рыбу */ }
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :

```



```
};
```

```
=====
РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)
=====
```

```

ФАЙЛ 74/87: src/components/visualizers/PrefixSumViz.tsx
```

```
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 411 | РАЗМ: 10.5 КБ

```

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from '../../../data/vizStepBus';
```

```
/**
```

```
 * Префиксные суммы: 1D и 2D.
```

```
 *
```

```
 * Главная фишка – наведение на число:
```

```
 * * в 1D наводим на P[i] – подсвечивается кусок массива
```

```
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
```

```
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
```

```
 * а при выбранном запросе показывается формула включения-исключения
```

```
 */
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];
```

```
};
```

```
/* ----- ID -----
```

```
const PrefixID: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]],
 []
);
};
```

```
/** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a" }>({ kind: "a" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
 : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
 : null;
```

```
const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;
```

```
const sum = pref[range.r + 1] - pref[range.l];
```

```
return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
```

```
</div>
```

```
{/* массив P */}
```

```
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === range.l;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue === null || liveValue === 0);
 const masked = !hasLiveP && shownDriven !== null;
 const isDrivenCur = shownDriven !== null && i === shownDriven;
 const shownValue = hasLiveP ? liveValue : null;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref", value: v })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 isDrivenCur
 ? "bg-emerald-600 text-white ring-1 ring-emerald-600"
 : blank
 ? "bg-slate-950 text-slate-600"
 : masked
 ? "bg-slate-800/50 text-slate-700"
 : active
 ? "bg-emerald-500 text-white"
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-500"
 }`}
 title={blank || masked ? `P[${i}] — null` : `P[${i}] = ${shownValue}`}>
 {blank || masked ? "." : String(shownValue)}
 </div>
)
)
 })}
```

```

 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-indigo-400">у
 у <b className="text-indigo-400">a – ку,
 </p>
)}
 </div>

 {/* Запрос */}
 <div className="bg-slate-900 border border-indigo-400 border-radius-10px p-10">
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву)
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-300">
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1}..{range.l}]
 {pref[range.r + 1]}
 {pref[range.l]}
 {range.l - range.r}
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) по префикс-сумме
 </p>
 </div>
</div>
);
};

```

/\* ----- 2D -----

```

const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);

```

```

 if (i === rect.r2 + 1 && j === rect.c2 + 1) return
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
 };

 return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 /* Исходная матрица */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
)
 })
 </div>
)
 </div>
 </div>
 </div>
)

```

```

 >
 {blank ? "." : String(shownValue)}
 {corner && !isHover && {
 <span className="absolute -top-
ht">
 {corner}

 }}
 </div>
 });
 }}}
</div>
 }}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 P[{hover.i}][{hover.j}] = {P[hover.i][hov
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор
 }}
</div>

<div className="bg-slate-900 border border-indigo
 <p className="font-mono text-sm sm:text-base te
 сумма = A<
 C + <s
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычтенный

```

```

for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j) - 1][j - 1]);
 }
}

```

// Data for 2D Matrix (Sparse Table 2D)

```

const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

```

```

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

```

```

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

```

```

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {

```

```

 className={`px-3 sm:px-4 py-2 rounded-lg text
 digo-600 text-white" : "bg-slate-800 text-s
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
</div>

<AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
</AnimatePresence>
</div>
);
};

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>();
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');

```



```

 onClick={() => setStep(Math.max(0, step - 1)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1)}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" >
 <div className="text-[11px] uppercase tracking-wider" >
 <div className="flex gap-1 sm:gap-1.5 overflow-x-hidden" >
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 const display = v === undefined || v === null
 ? "."
 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col items-center justify-center" >
 <div
 className={`flex items-center justify-center w-[44px] h-[44px] text-[clamp(26px,8vw,44px)] text-white
 inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-white"
 `
 >

```

```

</div>
<div className="text-center font-bold text-
rap">
 2^0 (L=1)
</div>
<div className="text-center font-bold text-
rap">
 2^1 (L=2)
</div>
<div className="text-center font-bold text-
rap">
 2^2 (L=4)
</div>
<div className="text-center font-bold text-
rap">
 2^3 (L=8)
</div>

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mon
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const liveRow = vizArray(liveRows?.[i]
 const liveValue = liveRow?.[j];
 // Уровень j=0 – это исходный a[i], о
 // очищаемого рабочего массива и не д
 const sourceValue = j === 0 ? liveSou
 const runtimeValue = liveValue ?? sou
 const runtimeHasValue = runtimeValue
 const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex(
 const isVisible = compilerLinked ? run

```

```

 // При st = [] каркас уже с
 opacity: compilerLinked ? 1
 scale: compilerLinked || is
 }}
 className={`w-[clamp(30px,9vw,
clamp(11px,3vw,17px))] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-v
 : isSrc2
 ? "bg-amber-500 text-v
 : isVisible
 ? "bg-slate-800 tex
 : compilerLinked
 ? "bg-slate-950 t
 : "bg-transparent
 `}
 title={compilerLinked ? `st[$
>
 {displayValue}
</motion.div>
)}

```

```

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-e
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",

```

```

 = min(
 <span className="text-emerald-400 font-b
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);
))()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: n
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);
 const liveR = vizNumber(vars?.r);

```

```

 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
 }}}
</div>

```

```

<div className="flex flex-col items-center gap-
rollbar">

```

```

 [...Array(shownK + 1)].map((_, i) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

```

```

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>

```

```

 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
 rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>

```

```

 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
 go-900/40 shadow-[0_0_25px_rgba(99,102,241,
 eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

```

```

 const liveValue = liveTable?.[`
 // А и st2 – разные объекты: по
 // слой тоже пуст. Значения А п
 // явной инициализации st2[(r,c
 const shownValue = compilerLink

```

```

 return (
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {
 if(locked && locked.
 else setLocked({r, c
 }
 }}
 className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
 title={hasLiveTable ? `st2[
 >
 {isBlank ? "." : String(sho
 </div>
);
 }}}
 </div>
 </div>
</div>
</React.Fragment>
)
 }}}
</div>
</div>

```

```

<div className="flex-1 min-w-0 flex flex-col gap-
 <div className="bg-slate-900 p-4 sm:p-6 rounded
 <h3 className="text-lg font-bold text-slate-3
 {shownK === 0 ? {
 <div className="text-slate-400 text-sm lea
 {liveTableEmpty ? {
 <div className="rounded-xl border bord
 <p className="font-bold text-amber-3
 <p className="mt-2 text-xs text-slat

```



```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

 // Координаты запроса – те же захардкоженные имена,
 useEffect(() => {
 const vars = runtime?.variables;
 const nr1 = vizNumber(vars?.r1);
 const nc1 = vizNumber(vars?.c1);
 const nr2 = vizNumber(vars?.r2);
 const nc2 = vizNumber(vars?.c2);
 if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
 if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
 if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
 if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
 }, [runtime]));

 const H = Math.max(1, r2 - r1 + 1);
 const W = Math.max(1, c2 - c1 + 1);
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6"
 <div className="flex-1 min-w-0 bg-slate-900"
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>
 <div className="flex flex-col items-cent

```





```
">r2 - Lx + 1][<span className="text
{'}}')});
</div>
```

```
<div className="w-full flex justify-cent
<div className="flex flex-col items-ce
<h4 className="text-slate-400 font-bo
Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{kx}][{ky}
</h4>
```

```
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`>
{Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky].toString();
```

```
 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white"; }
 else if (isTR) { color = "bg-lime-300 text-white"; }
 else if (isBL) { color = "bg-emerald-300 text-white"; }
 else if (isBR) { color = "bg-amber-300 text-white"; }

 return (
 <div key={idx} className={color}
 transition-all duration-300 relative shrink-0 ${c === c1 ? 'w-1/2' : 'w-1/2'}
 {txt}
 {badge && <div className="text-sm font-medium text-white">{badge}</div>}
```

```
)
 </div>
}
</div>
```

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header main');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

```

```

function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove('-translateX-100%');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('opacity-0');
 // Timeout allows display:block to be applied
 setTimeout(() => drawerOverlay.classList.remove('opacity-0'), 0);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('-translateX-100%');
 if (drawerOverlay) {
 drawerOverlay.classList.add('opacity-100');
 setTimeout(() => drawerOverlay.classList.add('opacity-100'), 0);
 }
 document.body.style.overflow = "";
 }
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => toggleDrawer());
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => toggleDrawer());
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => toggleDrawer());
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains('closed')) {
 toggleDrawer();
 }
 });
});

```

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.1/es5/tex-mml-chtml.js"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 },
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?family=Fira+Code&family=Arial+Black&display=block');
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; text-align: center;
 color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
 font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute;
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 0.8em; }
```

```

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important; }
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light

```

```

 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
 }
 .creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setMode('normal')" id="btn-normal" class="text-white
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf" class="text-white
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setTheme('dark')" id="btn-dark" class="text-white
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light" class="text-white
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook" class="text-white
 transition-colors">Тетрадь</button>
 </div>
 </div>
 </div>

```

```

 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2"
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1"
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform

```



```

 <a href="#section-1-2" class="b
 <a href="#section-1-3" class="b
 <a href="#section-1-4" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-2" class:
er-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-2-1" class="b
 <a href="#section-2-2" class="b
 <a href="#section-2-3" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-3" class:
er-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-3-1" class="b
 <a href="#section-3-2" class="b
 <a href="#section-3-3" class="b
 <a href="#section-3-4" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-4" class:
```

```


</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer hover-violet-500 pl-3 font-bold text-violet-300">
 7. Взаимно простые числа

</summary>
<div class="pl-6 space-y-2 text-violet-500">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer hover-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля Вейля

</summary>
<div class="pl-6 space-y-2 text-violet-500">


```

```


 11. НОД и НОК многочленов

 </summary>
 <div class="pl-6 space-y-2 text-cyan-500">
 1. НОД
 2. НОК
 3. НОД и НОК
 4. НОД и НОК
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer border-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые многочлены
 </summary>
 <div class="pl-6 space-y-2 text-blue-500">
 1. Взаимно простые многочлены
 2. Взаимно простые многочлены
 3. Взаимно простые многочлены
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer border-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов
 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">
 1. Корни многочленов
 2. Корни многочленов
 3. Корни многочленов
 </div>
</details>

```

```
<details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

</summary>
<div class="pl-6 mt-2 space-y-2">

</div>
</details>

<div class="mt-4 mb-2 text-xs font-l">


```

```

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting)
 visibleId = entry.target.id;

 // Highlight active
 document.querySelector(
 `a[href="#${entry.target.id}"]`
).style.fontWeight = 'bold';
 if (a.href.endsWith('#'))
 a.style.fontWeight = 'normal';
 });

 // Open the parent
 const correspondingLi = document.querySelector(
 `li[aria-labelledby="#${entry.target.id}"]`
);
 if (correspondingLi) {
 const details = correspondingLi.querySelector(
 'div[role="details"]'
);
 if (details && !details.open) {
 document.querySelector(
 `div[role="details"]`
).if (otherDetails) {
 otherDetails.open = false;
 }
 details.open = true;
 }
 }
 });
});

}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});

```

```

textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

```

```

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = 'white';
copyBtn.style.color = 'black';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(contentRaw);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = 'white';
closeBtn.style.color = 'black';
closeBtn.style.border = 'none';

```

```

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }
 const blob = new Blob([text], {
 type: 'text/markdown'
 });
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.md';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 {
 color: #000000;
 }
 `;
 }
 }

```

```

 body.bg-white table tr.
 body.bg-white table tr.
 body.bg-white .text-tea
 body.bg-white .text-rosa
 body.bg-white .text-emea
 body.bg-white .text-cyan
 body.bg-white .text-amber
 body.bg-white .text-fuchsia
 body.bg-white .text-lime
 body.bg-white .text-blue
 body.bg-white .text-green
 body.bg-white .text-indigo
 `;
} else if (theme === 'terminal')
 body.classList.add('bg-black')
 document.getElementById('theme')
 style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: #191919;
 }
 body.bg-black section h1,
 body.bg-black button {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 stroke: #22c55e !important;
 fill: transparent !important;
 `;
}

document.head.appendChild(style)

```



```
 -center justify-center text-green-400 hover:
 терминала"> </button>
 </div>
 </div>
 </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16
-base min-w-0">
 <header class="text-center mb-12 border-b border-white pb-4">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Высшая алгебра: теория и практика

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center">
 <h2 class="font-bold text-xl mb-4">Содержание
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Декомпозиция
 7. Алгебраические структуры
 8. Кольца
 9. Коле
 10.
 11.
 12.
 13.
 14.

 </div>
</main>
```

```

 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 2: Базовые операции / Евклид / НОД и НОК

-4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 3: НОД и НОК / Алгоритм Евклида

-4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структурная

-4 border-yellow-500 hover:bg-slate-700 transition-colors

```

```

<a href="#question-15" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-5
 <div class="font-bold text-slate

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emeral
 <div class="font-bold text-slate

 <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-5
 <div class="font-bold text-slate

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo
 <div class="font-bold text-slate

 <a href="#question-22" onclick="setl

```

```

<!-- Прямое -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white; padding: 10px;">
 <h3 class="text-xl text-blue-400">Прямое
 <p class="text-sm text-slate-400">Прямое
 <ul class="list-disc pl-5 sp-4">

 k$).

 ово.

 ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white; padding: 10px;">
 <h3 class="text-xl text-fuchsia-400">От противного
 <p class="text-sm text-slate-400">От противного
 <ul class="list-disc pl-5 sp-4">

 i>

 т.д.).

 что так и задумано: "Ой, противоречие, теорема не верна".

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white; padding: 10px;">
 <h3 class="text-xl text-emerald-400">Единственность
 <p class="text-sm text-slate-400">Единственность
 <ul class="list-disc pl-5 sp-4">

 , r_2$).

 счет ограничений (размер остатка) докажи, что

```

```
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
со следующим. Если в конце 0 – пациент мертв.
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
c. Пока получается 0 – корень жив. Как то.
корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
ициентов (они делятся на p), кроме старшего
зложить.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
. Чаще всего после этого Фейс-контроль p.
 </div>
</div>
```

```
 <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">
```

```
 </div>
 </section>
</div>
```

```
<div id="questions-container">
```

```
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 78/87: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 79/87: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
```

```
/**
```

```
 * Выгрузка главы (или всего пособия) в самостоятельный
```

```
 *
```

```
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid #0f172a; }
.export-gloss h2 { color: #fff; font-size: 1.25rem; font-weight: bold; }
.export-term { background: #0f172a; border: 1px solid #0f172a; padding: 10px; }
.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; font-weight: bold; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #0f172a; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на глоссарий
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```
 try {
```

```
 annotateTerms(host);
```

```
 } catch {
```

```
 /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
 }
```

```
 const ids: string[] = [];
```

```
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
```

```
 const id = el.dataset.termId;
```

```
function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercas
 <h1 style="color:#fff;font-size:1.6rem;font-weight:
 <div style="font-size:.72rem;color:#475569;margin-t
 </header>

 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(op
 </div>

 <article class="prose prose-invert max-w-none">
${opts.body}
 </article>

${opts.gloss}

 <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```



```

const bodies: string[] = [];

const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#c
 .join("\n");

chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style=
<p style="text-align:right;font-size:.72rem;margin:0 0 :
 });

const body = `<nav id="toc" style="background:#0f172a
 rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:8
 <ol style="margin:0;padding-left:1.1rem;font-size:.85
</nav>
`${bodies.join("\n")}`;

const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
});
triggerDownload(html, "posobie-vse-temy.html");
}

```

```

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG | image |");
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
);
 }
} else {

```

```

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
 .replace(/<[^>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }
}

```

```
 .filter((f) => f.endsWith(".png"))
 .sort();

if (pages.length === 0) throw new Error("Не найдено н

ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный к
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width:
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(` выход: ${path.relative(ROOT, PDF_P
}
}
```

```
[/^src\/data\/$/, "Контент и данные пособия"],
[/^src\/components\/visualizers\/$/, "Визуализаторы (в"],
[/^src\/components\/$/, "Интерактивные компоненты и сим"],
[/^src\/layout\/$/, "HTML-разметка (layout)"],
[/^src\/utils\/$/, "Утилиты"],
[/^src\/$/, "Ядро приложения"],
[/^scripts\/$/, "Скрипты сборки и экспорта"],
[/^\.github\/$/, "CI / автоматизация"],
[/^[^\/]+$/, "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];
```

```
function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 });
 files
 .split("\n")
 .filter(Boolean);
 } catch {
```

```

 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 }));
}

```

```

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:        ${chapters.length}`)
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
 push(HR);
 chapters.forEach((c, i) => {
 push(` ${String(i + 1).padStart(3, " ")} ${c.titl
 });
 push();
}

```

```

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

---

ФАЙЛ 84/87: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

---

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),

```

```
for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
}
throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
");
}
```

---

ФАЙЛ 85/87: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.5 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT,
 PAGES_DIR,
 TXT_PATH,
 PAGE,
 ensureDir,
 humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);
```



```
const jobs = pages.map(({lines, idx} => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
}));
```

```
const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];
```

```
const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;
```

```
async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
```

```
Если там «Deploy from a branch» (legacy) – шаг Deploy
2. Settings → Environments → github-pages → Deployment
добавьте паттерн «arena/**» и ветку «main» (или main/*)
иначе деплой из новой рабочей ветки отклоняется.
```

```
on:
 push:
 branches:
 - main
 # Позволяет проверить Pages прямо из рабочей ветки
 - "arena/**"
```

```
workflow_dispatch:
```

```
permissions:
 contents: read
 pages: write
 id-token: write
```

```
concurrency:
 group: pages
 cancel-in-progress: true
```

```
jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund
```

```

 uses: actions/deploy-pages@v5

```

ФАЙЛ 87/87: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0 КБ

```

name: Rebuild code PDF
```

```
Пайплайн пересборки PDF при каждом коммите:
```

```

исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (PDF)
|
→ full_code_all_page.pdf (PDF)
#
```

```
Готовые TXT и PDF коммитаются обратно в ветку (их отдаёт GitHub Actions)
```

```
промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - "**"
```

```
 paths-ignore:
```

```
 # чтобы коммит самого workflow не запускал бесконечно
```

```
 - "public/export/**"
```

```
 - "**/*.md"
```

```
 workflow_dispatch:
```

```
 inputs:
```

```
 density:
```

```
 description: "DPI растеризации страниц (по умолчанию 300)"
```

```
 required: false
```

```
 default: "150"
```

```
concurrency:
```

```
 group: rebuild-pdf-${{ github.ref }}
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
  {  
    echo "### Экспорт пересобран"  
    echo ""  
    echo "| Артефакт | Размер |"  
    echo "| --- | --- |"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| PNG-страниц | \$(ls tmp/export-pages.  
  } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-export  
  path: |  
    public/export/full\_code\_all\_pages.pdf  
    public/export/full\_code\_all\_pages.txt  
  if-no-files-found: error